

第三篇 现代网页布局

Contents

01. 模式转换

display

02. flex布局

flex

03. grid布局

grid

04. 定位布局

position

05. 多列布局

column

CSS 布局

Finally here~

CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



CSS 布局

Finally here~

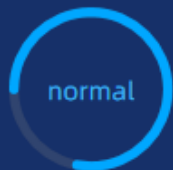
CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



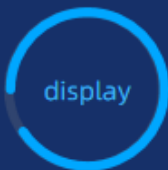
CSS 布局

Finally here~

CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



正常布局



模式转换布局



弹性布局



网格布局



定位布局



多列布局

每种技术都有它们的用途，各有优缺点，相互辅助。

- 简单布局：优先使用 Flexbox（一维）或 Grid（二维）。
- 复杂响应式布局：使用 Grid + 媒体查询。
- 文本内容分栏：多列布局（column-count）
- 兼容旧浏览器：浮动布局 或 Flexbox 的降级方案。
- CSS Grid 逐渐成为主流，支持更复杂的布局场景。

1. CSS正常布局流

normal flow

正常布局流（normal flow）是指在不对页面进行任何布局控制时，浏览器默认的 HTML 布局方式。也称为**标准流**。
正常布局流是 CSS 布局的基石，页面大的布局基本就是利用区块元素**上下罗列**而成。

区块元素(block)

- 独占一行，宽度默认撑满父容器。
- 垂直方向排列，可设置宽高。
- 示例：div、p、h1等。

行内元素(inline)

- 水平方向依次排列，直到容器宽度不足则换行。
- 宽度和高度由内容决定，无法直接设置。
- 示例：span、img、strong 等

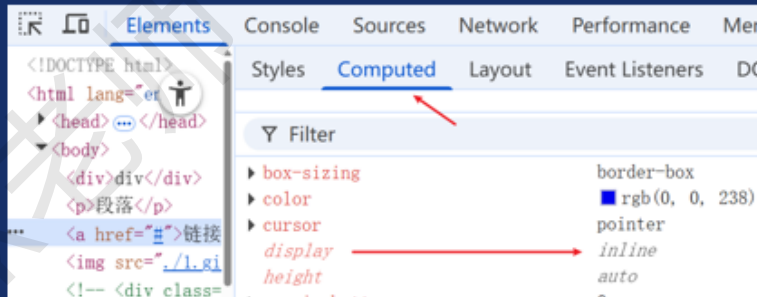
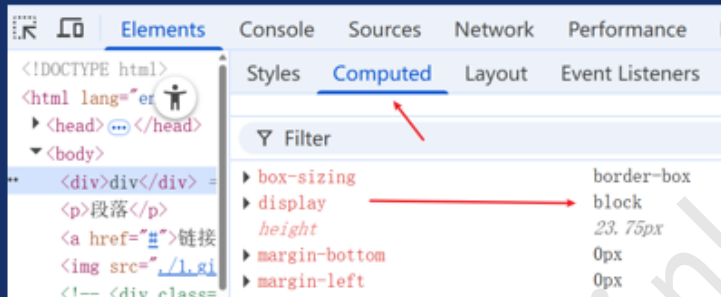
文档流方向

默认从上到下、从左到右排列。

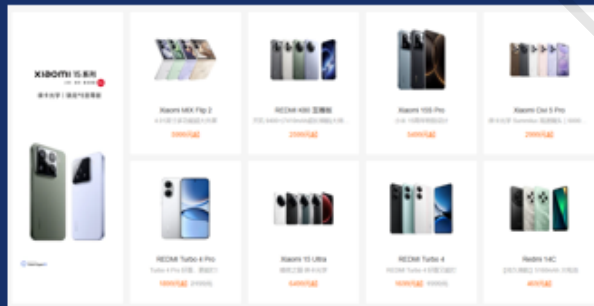
display 显示属性

display

正常流中的所有内容都有一个 `display` 的属性，用作元素的默认行为方式



但是开发的时候，我们需要一个块级盒子一行排列，或者行内元素需要设置大小，怎么办？



2. display 转换

display

display 属性允许我们更改默认的显示方式此属性。

display: block; 转换为区块元素（常用）

display: inline; 转换为行内元素（罕见）

display: inline-block; 转换为行内块元素（较少）

属性值	是否独占一行	能否设置宽高	默认宽度
display: block	是	☒	撑满父容器宽度
display: inline	否	✗	由内容决定
display: inline-block	否	☒	由内容决定（可覆盖）

2. display 转换

display

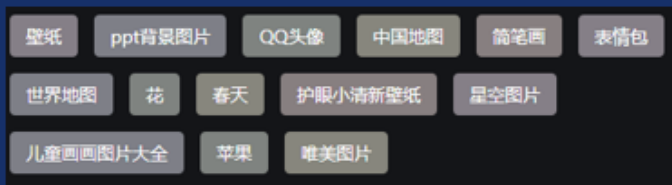
表单元素默认 inline-block，其他元素想要转换可以：`display: inline-block;`

使用场景：

1. 让块级盒子一行显示。
2. 让行内盒子具备宽度高度。

注意：

1. 行内块元素中间会有空白缝隙。给父元素字号改为0可以去掉。
2. 实际开发适合于对间距要求不高的情况下可以转换。
3. 如果真的要精细布局，请用 flex 或者 grid 更合适。



总结

gen zhe wo~

- 手机 >
- 电视 >
- 家电 >
- 笔记本 平板 显示器 >
- 出行 穿戴 >
- 耳机 音箱 >
- 健康 儿童 >
- 生活 箱包 >

探索vivo



1. display 模式转换

属性值	是否独占一行	能否设置宽高	默认宽度
display: block	是	☒	撑满父容器宽度
display: inline	否	☒	由内容决定
display: inline-block	否	☒	由内容决定（可覆盖）

行内块元素中间会有空白缝隙。给父元素字号改为0可以去掉

壁纸

ppt背景图片

QQ头像

中国地图

简笔画

表情包

世界地图

花

春天

护眼小清新壁纸

星空图片

儿童画画图片大全

苹果

唯美图片

2. display 转换

display

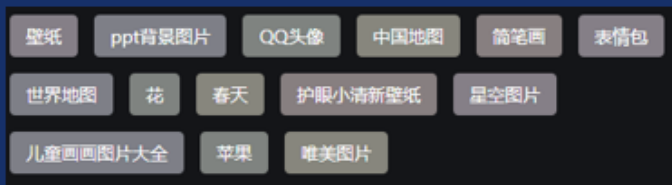
表单元素默认 inline-block，其他元素想要转换可以：`display: inline-block;`

使用场景：

1. 让块级盒子一行显示。
2. 让行内盒子具备宽度高度。

注意：

1. 行内块元素中间会有空白缝隙。给父元素字号改为0可以去掉。
2. 实际开发适合于对间距要求不高的情况下可以转换。
3. 如果真的要精细布局，请用 flex 或者 grid 更合适。



3. 浮动

float

浮动(float) 最早是做“文字环绕”效果的。



刘德华（Andy Lau），1961年9月27日出生于中国香港，华语影视男演员、流行乐歌手、电影制片人、作词人。1981年，出演首部电影《彩云曲》并进入演艺圈。1985年，发行首张专辑《只知道此刻爱你》。1990年，凭借专辑《可不可以》在歌坛获得关注；同年，主演陈木胜执导的爱情片《天若有情》。1992年，发行的粤语专辑《真我的风采》在中国香港取得40万张销量。1993年，首次在红馆举行个人演唱会。

1994年，获十大劲歌金曲最受欢迎男歌星奖 [175]。1995年，登上央视春晚并演唱歌曲《忘情水》 [339]。2000年，凭借警匪片《暗战》获第19届香港电影金像奖最佳男主角奖 [320]；同年，发行音乐专辑《男人的爱》 [235]，并被《吉尼斯世界纪录大全》评为“获奖最多的香港男歌手” [7]。2002年，主演的警匪片《无间道》成为其电影生涯代表作

3. 浮动

float

浮动(float) 最早是做“文字环绕”效果的。



刘德华 (Andy Lau)，1961年9月27日出生于中国香港，华语影视男演员、流行乐歌手、电影制片人、作词人。1981年，出演首部电影《彩云曲》并进入演艺圈。1985年，发行首张专辑《只知道此刻爱你》。1990年，凭借专辑《可不可以》在歌坛获得关注；同年，主演陈木胜执导

的爱情片《天若有情》。1992年，发行的粤语专辑《真我的风采》在中国香港取得40万张销量。1993年，首次在红馆举行个人演唱会。

1994年，获十大劲歌金曲最受欢迎男歌星奖 [175]。1995年，登上央视春晚并演唱歌曲《忘情水》 [339]。2000年，凭借警匪片《暗战》获第19届香港电影金像奖最佳男主角奖 [320]；同年，发行音乐专辑《男人的爱》 [235]，并被《吉尼斯世界纪录大全》评为“获奖最多的香港男歌手” [7]。2002年，主演的警匪片《无间道》成为其电影生涯代表作

给图片添加 `float: left;` 样式

3. 浮动

float

浮动(float) 可以让元素脱离文档流，向左或向右浮动，直到碰到父容器边缘或其他浮动元素

属性值	作用	示例
left	左侧浮动	float: left;
right	右侧浮动	float: right;
none	默认值，不浮动	float: none;



3. 浮动

float

浮动(float) 可以让元素脱离文档流，向左或向右浮动，直到碰到父容器边缘或其他浮动元素

属性值	作用	示例
left	左侧浮动	float: left;
right	右侧浮动	float: right;
none	默认值，不浮动	float: none;

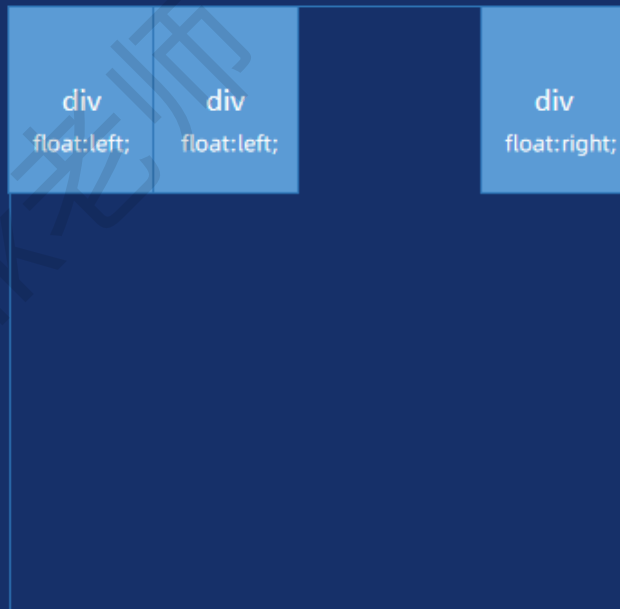


3. 浮动

float

浮动(float) 可以让元素脱离文档流，向左或向右浮动，直到碰到父容器边缘或其他浮动元素

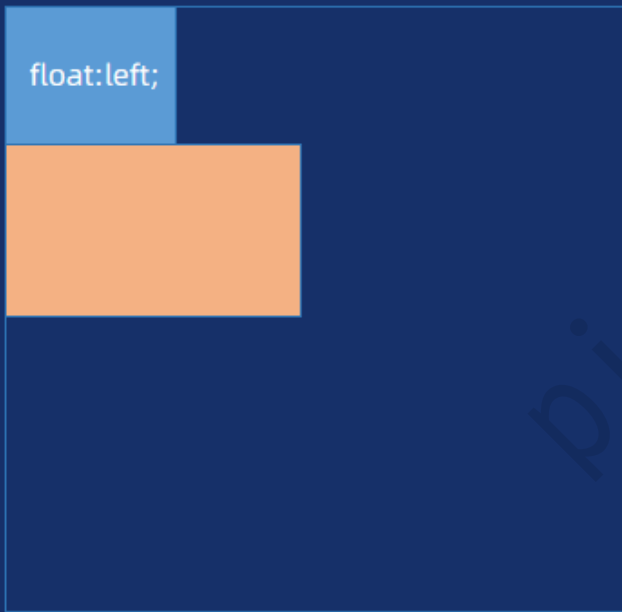
属性值	作用	示例
left	左侧浮动	float: left;
right	右侧浮动	float: right;
none	默认值，不浮动	float: none;



3. 浮动

float

浮动(float) 可以让元素脱离文档流，影响周围元素的布局。



3. 浮动

float

浮动(float) 可以让元素脱离文档流, 影响周围元素的布局。



脱离文档流: 盒子上浮其他文档流(标准流)上, 浮动的盒子不占位置。

3. 浮动

float

浮动(float) 可以让元素脱离文档流，影响周围元素的布局。



浮动带来的影响：

1. 父盒子没有高度。（很多情况下不能给父亲指定高度）
2. 子元素浮动。
3. 影响其他盒子布局了。

清除浮动也可以理解为闭合浮动，简单来说，就是让浮动的元素尽量控制在父盒子内，不要影响其他盒子。

3. 浮动-清除浮动

float

清除浮动主要有四种方式：



额外标签法

在浮动元素的最后面，
新增一个块级标签。

添加属性：
`clear: both;`



单伪元素清除浮动

父元素添加伪元素。

比如： 京东

```
.clearfix:after {  
  visibility: hidden;  
  clear: both;  
  display: block;  
  content: ".";  
  height: 0  
}
```



双伪元素清除浮动

父元素添加双伪元素。

比如： 小米

```
.clearfix:after, .clearfix:before {  
  content: ".";  
  display: table  
}  
  
.clearfix:after {  
  clear: both  
}
```



overflow 清除浮动

父元素添加overflow。

4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的对齐、分布和空间分配。



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

弹性盒子核心:

1. 父控子 (亲父子)

- 父盒子控制子盒子如何排列布局



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

弹性盒子核心:

1. 父控子 (亲父子)

▣ 父盒子控制子盒子如何排列布局



4. 弹性盒子

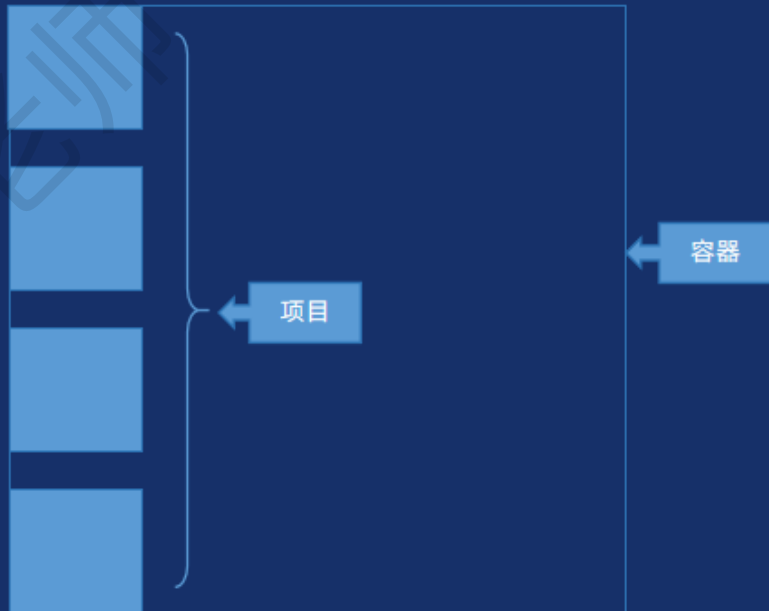
flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

弹性盒子核心:

1. 父控子 (亲父子)

- ❑ 父盒子**控制**子盒子如何排列布局
- ❑ 父盒子称为**容器**, 子盒子称为**项目**



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的对齐、分布和空间分配。

弹性盒子核心:

1. 父控子 (亲父子)

- ❑ 父盒子控制子盒子如何排列布局
- ❑ 父盒子称为容器, 子盒子称为项目

2. 主轴和交叉轴 (侧轴)

- ❑ 主轴默认水平方向, 交叉轴默认垂直方向, 可以更改

主轴



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

弹性盒子核心:

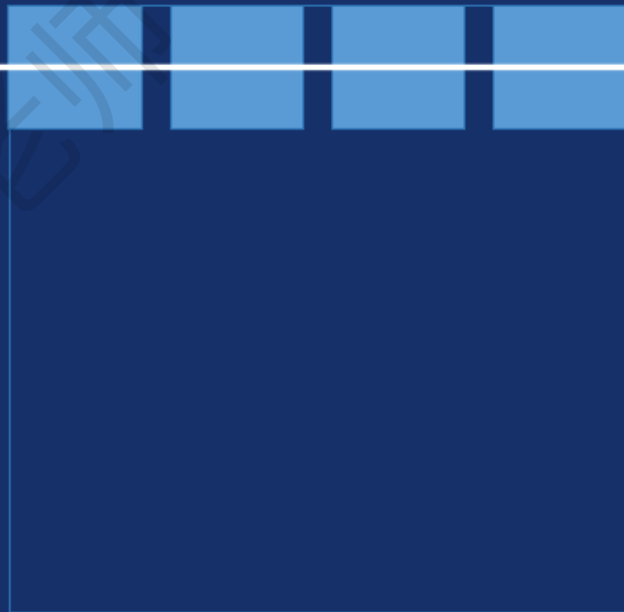
1. 父控子 (亲父子)

- ❑ 父盒子**控制**子盒子如何排列布局
- ❑ 父盒子称为**容器**, 子盒子称为**项目**

2. 主轴和交叉轴 (侧轴)

- ❑ 主轴默认水平方向, 交叉轴默认垂直方向, 可以更改

主轴



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

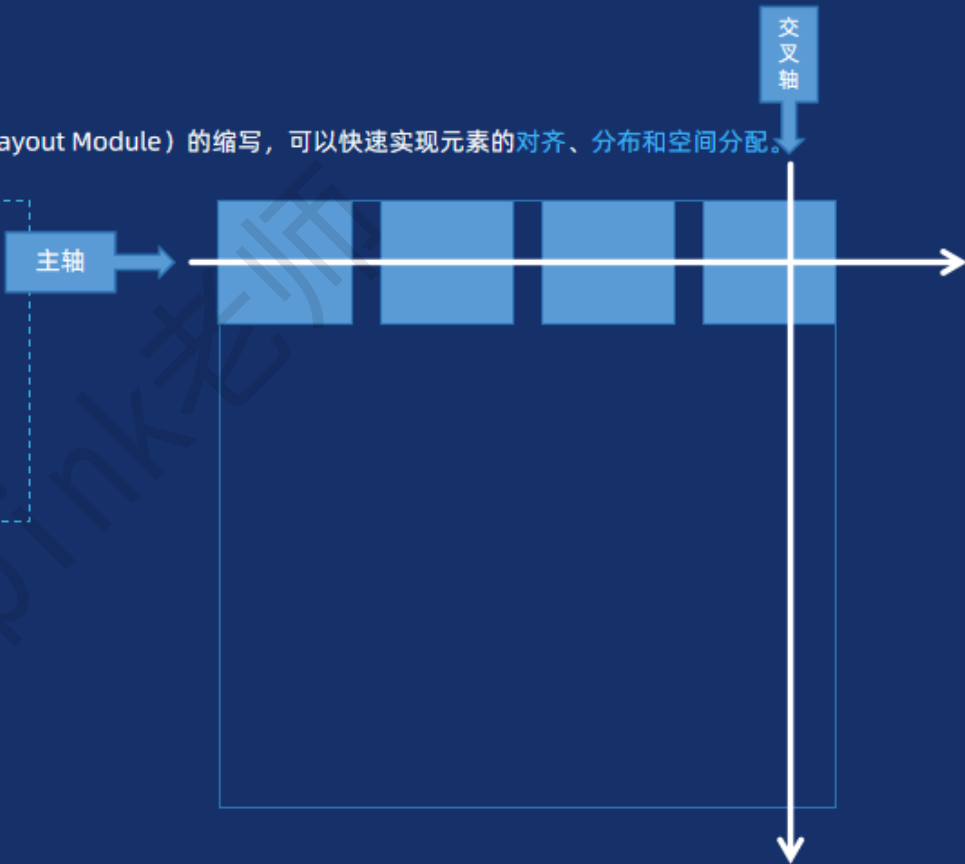
弹性盒子核心:

1. 父控子 (亲父子)

- 父盒子**控制**子盒子如何排列布局
- 父盒子称为**容器**, 子盒子称为**项目**

2. 主轴和交叉轴 (侧轴)

- 主轴默认水平方向, 交叉轴默认垂直方向, 可以更改



4. 弹性盒子

flexbox

Flexbox 是 CSS 弹性盒子布局模块 (Flexible Box Layout Module) 的缩写, 可以快速实现元素的**对齐、分布和空间分配**。

弹性盒子核心:

1. 父控子 (亲父子)

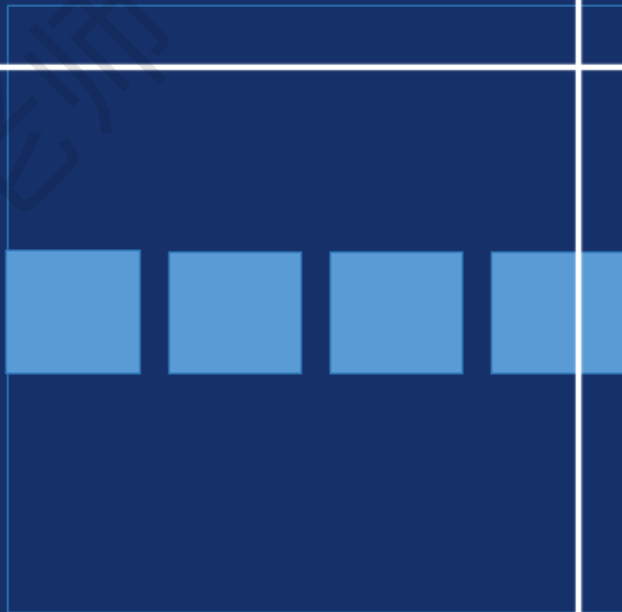
- 父盒子**控制**子盒子如何排列布局
- 父盒子称为**容器**, 子盒子称为**项目**

2. 主轴和交叉轴 (侧轴)

- 主轴默认水平方向, 交叉轴默认垂直方向, 可以更改

主轴

交叉轴



4. 弹性盒子

flexbox

容器（父盒子）属性

属性	作用	示例
display	定义元素为 Flex 容器	<code>.container { display: flex; }</code>
flex-direction	定义主轴方向（项目排列方向）	<code>.container { flex-direction: row; }</code>
flex-wrap	控制是否换行	<code>.container { flex-wrap: wrap; }</code>
justify-content	定义主轴上的对齐方式（整体分布）	<code>.container { justify-content: center; }</code>
align-items	定义交叉轴上的对齐方式（单行时整体对齐）	<code>.container { align-items: center; }</code>
align-content	定义多行时交叉轴上的对齐方式（仅当 flex-wrap: wrap 且内容换行时生效）	<code>.container { align-content: space-between; }</code>

4. 弹性盒子

flexbox

项目（子盒子）属性

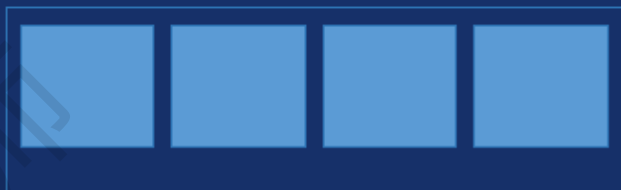
属性	作用	示例
order	定义项目的排列顺序（默认 0，数值越小越靠前）	<code>.item { order: -1; }</code>
flex-grow	定义项目的放大比例（默认 0，即不放大）	<code>.item { flex-grow: 1; }</code>
flex-shrink	定义项目的缩小比例（默认 1，空间不足时等比缩小）	<code>.item { flex-shrink: 0; }</code>
flex-basis	定义项目在主轴方向上的初始大小（优先级高于 width/height）	<code>.item { flex-basis: 200px; }</code>
flex	flex-grow、flex-shrink、flex-basis 的简写	<code>.item { flex: 1; }</code>
align-self	覆盖容器的 align-items，单独定义某个项目的交叉轴对齐方式	<code>.item { align-self: center; }</code>

4. 弹性盒子

flexbox

容器(父盒子)设置 `display: flex;` 可以让子盒子按照**主轴**方式排列。

- ❑ 如果子元素有大小，则按照**给定大小**来显示。
- ❑ 如果子元素没有大小，则**拉伸**充满父容器。
- ❑ 若子元素总宽度超过容器宽度，默认会**压缩**子元素。



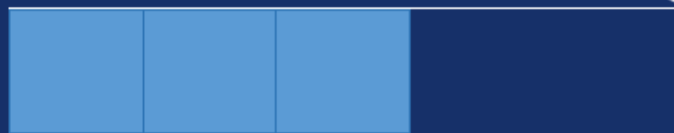
开发中大部分情况下，父容器设置`display: flex`，子元素设定大小。

4. 弹性盒子

flexbox

justify-content 定义主轴上的对齐方式

属性值	效果	示例
flex-start	左对齐 (默认)	子元素靠左排列
flex-end	右对齐	子元素靠右排列
center	居中对齐	子元素居中
space-between	两端对齐	首个子元素放置于起点 末尾元素放置于终点
space-around	项目两侧间隔相等	每个子元素周围分配相同的空间
space-evenly	项目间隔均匀分布	每个子元素之间的间隔相等



flex-start



flex-end

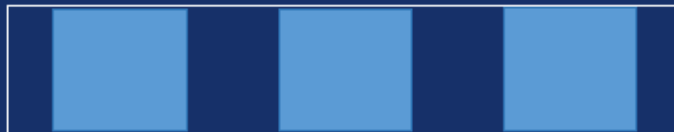


center



space-between

A--B--C



space-around

-A--B--C-



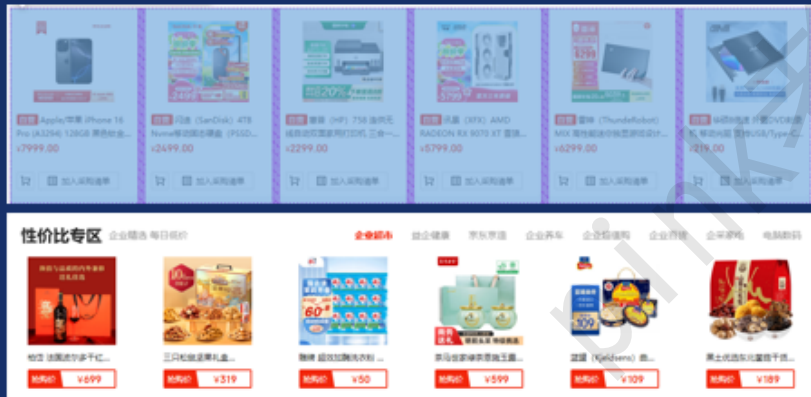
space-evenly

--A--B--C--

4. 弹性盒子

flexbox

justify-content 定义主轴上的对齐方式



space-between



space-around



space-evenly

4. 弹性盒子

flexbox



`align-items` 定义交叉轴上的对齐方式（单行时整体对齐）

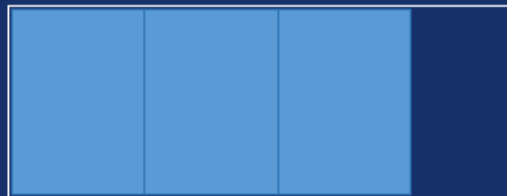
属性值	作用描述
<code>flex-start</code>	项目在交叉轴起点对齐（默认值）
<code>flex-end</code>	项目在交叉轴终点对齐
<code>center</code>	项目在交叉轴居中对齐
<code>stretch</code>	项目拉伸填充整个容器高度（需子项目无固定高度）



flex-start



center

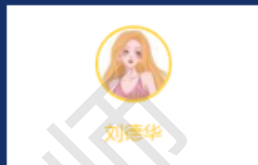


stretch

4. 弹性盒子

flexbox

flex-direction 定义主轴方向（改变主轴方向）

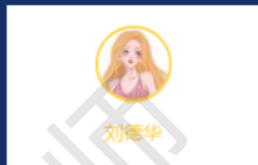
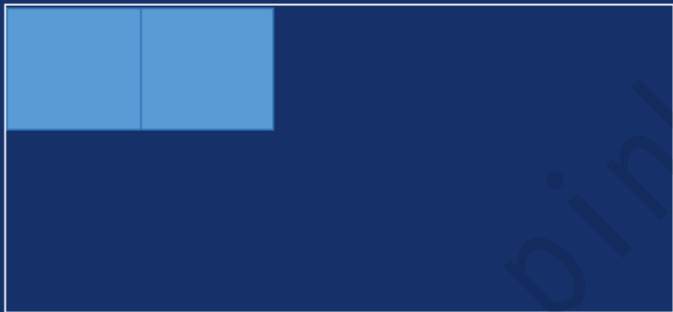


属性值	描述	示例效果	代码示例
row	默认值。子元素沿水平主轴（从左到右）排列。	A B C（横向排列）	<code>.container { flex-direction: row; }</code>
row-reverse	子元素沿水平主轴反向排列（从右到左）。	C B A（反向横向排列）	<code>.container { flex-direction: row-reverse; }</code>
column	子元素沿垂直主轴（从上到下）排列。	A B C（纵向排列）	<code>.container { flex-direction: column; }</code>
column-reverse	子元素沿垂直主轴反向排列（从下到上）。	C B A（反向纵向排列）	<code>.container { flex-direction: column-reverse; }</code>

4. 弹性盒子

flexbox

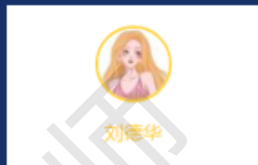
flex-direction 定义主轴方向



4. 弹性盒子

flexbox

flex-direction 定义主轴方向

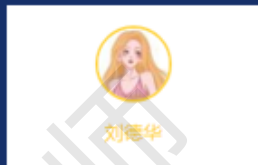


1. 改变主轴方向

4. 弹性盒子

flexbox

flex-direction 定义主轴方向

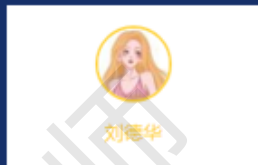
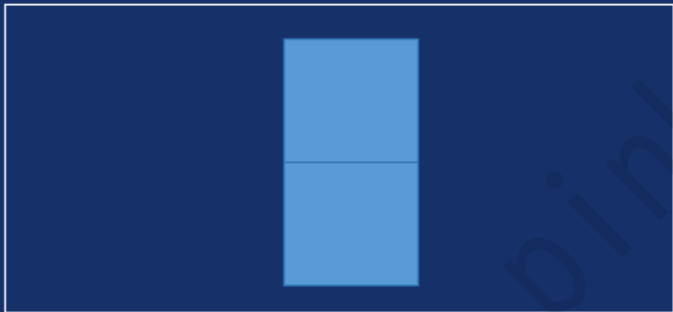


1. 改变主轴方向
2. 主轴居中对齐

4. 弹性盒子

flexbox

flex-direction 定义主轴方向



1. 改变主轴方向
2. 主轴居中对齐
3. 交叉轴居中对齐

4. 弹性盒子

flexbox

flex-wrap 控制是否换行

属性值	排列效果
nowrap	不换行 ABCDEFGH (全部横向排列, 可能被压缩)
wrap	ABCD EFGH (第一行下方, 第二行下方) 换行
wrap-reverse	EFGH ABCD (第一行下方, 第二行上方) 翻转 (了解即可)



4. 弹性盒子

flexbox

`align-content` 定义多行时交叉轴上的对齐方式（仅当 `flex-wrap: wrap` 且内容换行时生效）

属性值	效果	示例
<code>flex-start</code>	上对齐	子元素靠上排列
<code>flex-end</code>	下对齐	子元素靠下排列
<code>center</code>	居中对齐	子元素居中
<code>space-between</code>	两端对齐	首个子元素放置于起点 末尾元素放置于终点
<code>space-around</code>	项目两侧间隔相等	每个子元素周围分配相同的空间
<code>space-evenly</code>	项目间隔均匀分布	每个子元素之间的间隔相等



开发中大部分情况下，该属性使用较少，大部分子元素顺序排列即可。

4. 弹性盒子

flexbox

项目（子盒子）属性

子元素的属性用于控制自身的尺寸、顺序或对齐方式

语法：

`flex: 1;` 剩余空间占1份，并且可以伸缩盒子大小

数字表示剩余空间所占份数。正整数

4. 弹性盒子

flexbox

项目（子盒子）属性

子元素的属性用于控制自身的尺寸、顺序或对齐方式

属性	作用	示例
flex-grow	定义子元素剩余空间分配 放大比例 (默认 0, 即不放大)	.item { flex-grow: 1; }
flex-shrink	定义子元素剩余空间分配 缩小比例 (默认 1, 空间不足时等比缩小, 0 为不缩小)	.item { flex-shrink: 0; }
flex-basis	定义项目在主轴方向上的 初始大小 (默认 auto, 优先级高于 width/height)	.item { flex-basis: 200px; }
flex	flex-grow、flex-shrink、flex-basis 的简写 flex: 1 → 1 1 0% (等比放大/收缩, 初始无基准尺寸) flex: 2 → 2 1 0% (等比放大/收缩, 初始无基准尺寸)	.item { flex: 1; }
align-self	覆盖容器的 align-items, 单独定义某个项目的交叉轴对齐方式	.item { align-self: center; }
order	定义项目的排列顺序 (默认 0, 数值越小越靠前)	.item { order: -1; }

gap间距

gap

gap 简写属性用于设置行与列之间的间隙（间距）

使用场景：

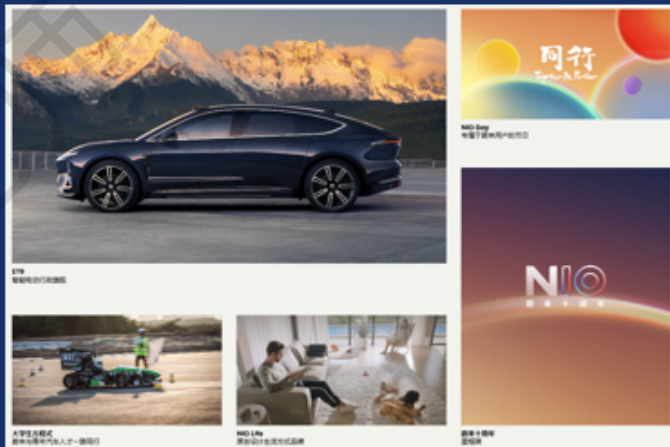
1. 让子元素之间保留间隙

语法：

gap: 20px; 行和列之间保持20像素间隙。

gap: 20px 30px; 行间距是20像素，列间距是30像素。

注意：gap是写到父元素（容器身上）



4. 弹性盒子

flexbox

项目（子盒子）属性

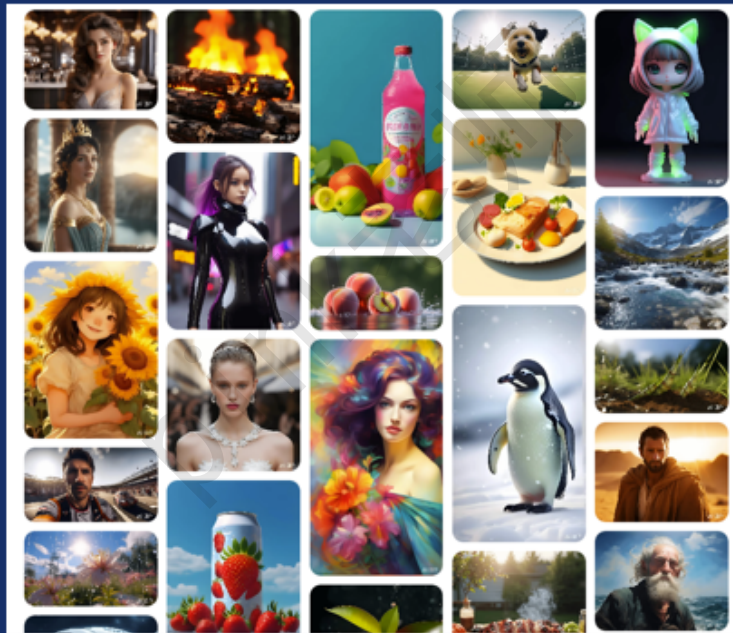
子元素的属性用于控制自身的尺寸、顺序或对齐方式

属性	作用	示例
flex-grow	定义子元素剩余空间分配 放大比例 (默认 0, 即不放大)	.item { flex-grow: 1; }
flex-shrink	定义子元素剩余空间分配 缩小比例 (默认 1, 空间不足时等比缩小, 0 为不缩小)	.item { flex-shrink: 0; }
flex-basis	定义项目在主轴方向上的 初始大小 (默认 auto, 优先级高于 width/height)	.item { flex-basis: 200px; }
flex	flex-grow、flex-shrink、flex-basis 的简写 flex: 1 → 1 1 0% (等比放大/收缩, 初始无基准尺寸) flex: 2 → 2 1 0% (等比放大/收缩, 初始无基准尺寸)	.item { flex: 1; }
align-self	覆盖容器的 align-items, 单独定义某个项目的交叉轴对齐方式	.item { align-self: center; }
order	定义项目的排列顺序 (默认 0, 数值越小越靠前)	.item { order: -1; }

弹性布局实战

ok let's go go go ~

案例： 百度图片



<https://image.baidu.com/>

垂直对齐-解决图片底部空白缝隙问题

vertical

基线：浏览器行内元素（行内块元素）排版中存在用于对齐的基线（baseline）



中文 English

顶线 (top)

中线 (middle)

基线 (baseline)

底线 (bottom)

图片默认和文字基线对齐

解决方案：

1. 把图片转换为块级元素。
2. 设置图片对齐方式不是基线对齐即可。

```
img {  
  vertical-align: top;  
}
```

属性值	效果
baseline	默认，基线对齐
top	顶部对齐
middle	中部对齐
bottom	底部对齐

垂直对齐-解决图片底部空白缝隙问题

vertical

让文字和图片垂直居中

❤️ 为你推荐

🛒 心选秒杀

📱 潮电好物

🪑 居家优品

🍉 超市百货

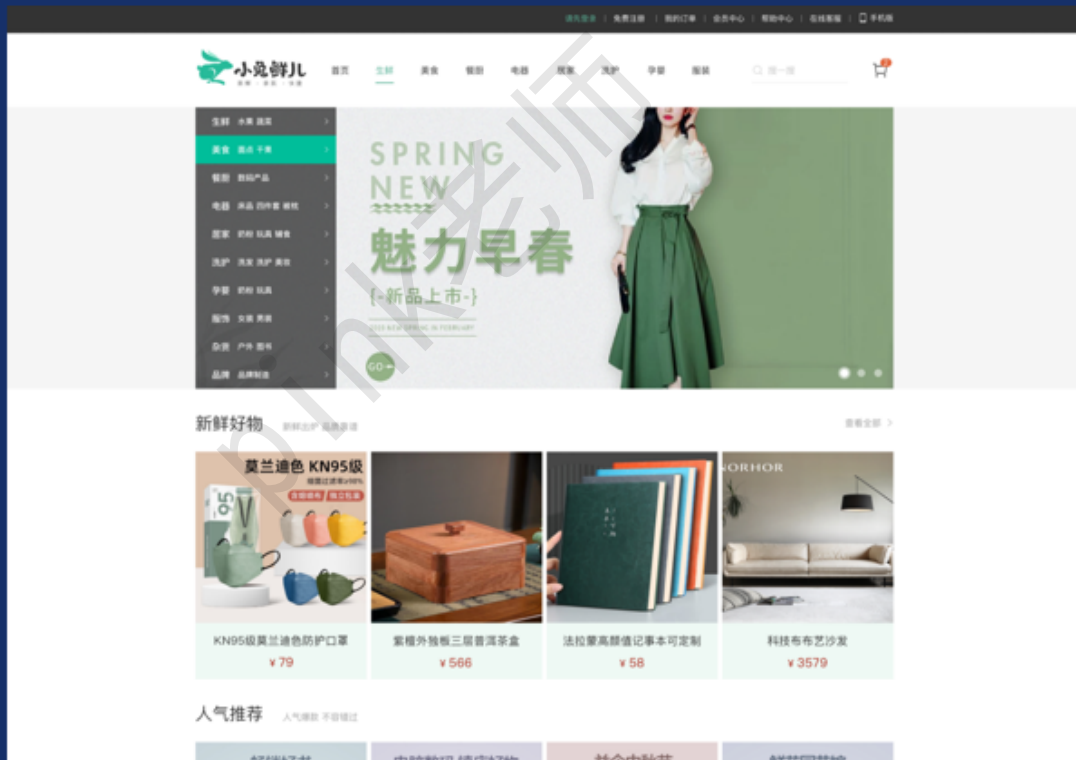
🛍️ 进口好物

```
img {  
  border: 0;  
  vertical-align: middle  
}
```

弹性布局实战

ok let's go go go ~

小兔鲜儿页面



弹性布局实战

ok let's go go go ~

案例：小兔鲜页面



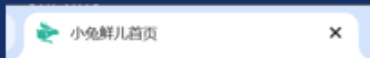
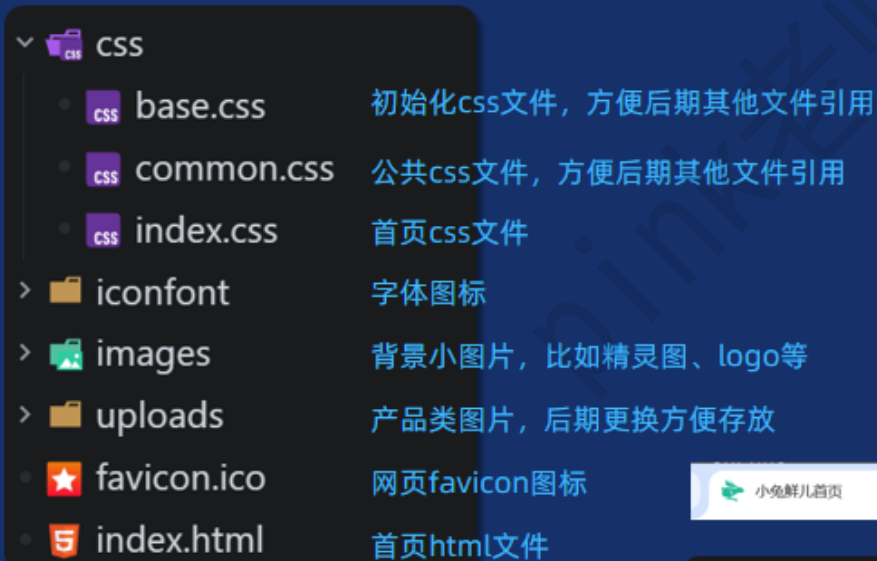
公共样式好处：

1. 多次使用，节约代码。
2. 方便维护，统一修改。

弹性布局实战

ok let's go go go ~

案例：小兔鲜页面



<!-- 网页图标 favicon图标 一定放到根目录下-->

```
<link rel="icon" href="./favicon.ico" type="image/x-icon">
```

弹性布局实战

ok let's go go go ~

案例：小兔鲜页面- logo做法



1. logo给用h1 提高权重
2. 里面包含[链接](#)，可以点击跳转到首页。
3. 给链接的[背景图片](#)。并且给链接添加title。
4. 链接里面写网站名字，把文字隐藏。
 - 可以利用text-indent: 负值，然后裁剪 overflow:hidden;

弹性布局实战

ok let's go go go ~

案例：小兔鲜页面- entry



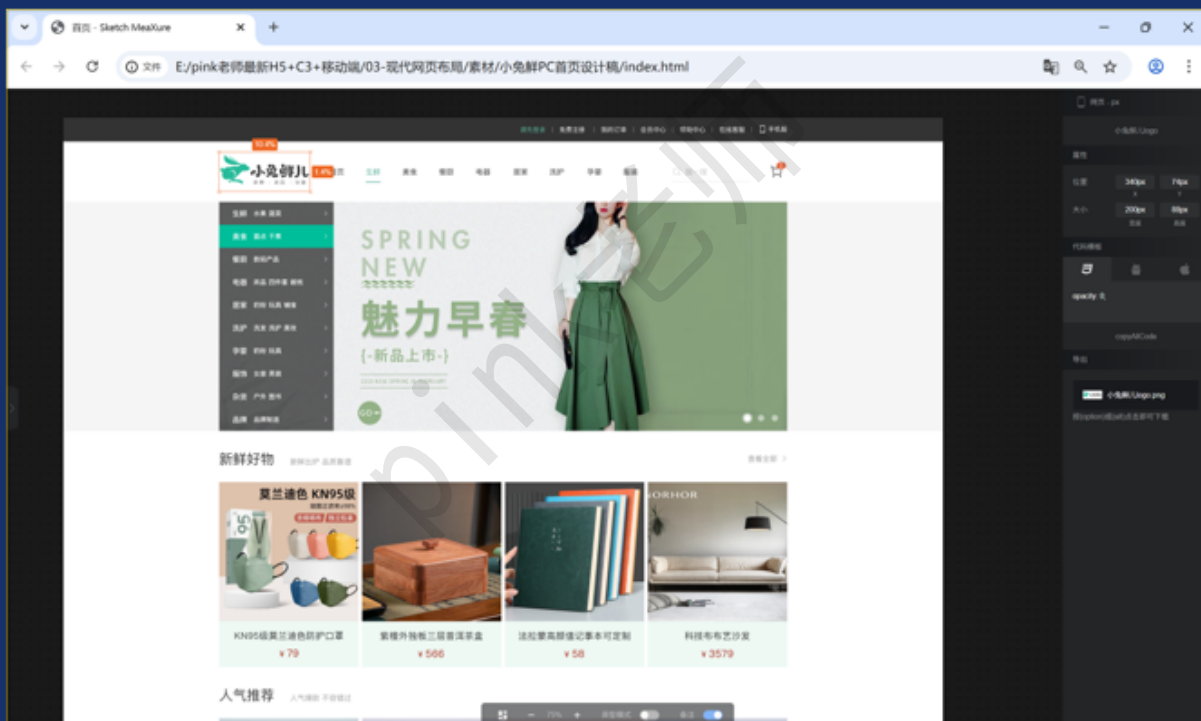
category

banner

小兔鲜实战总结

are you ok ~

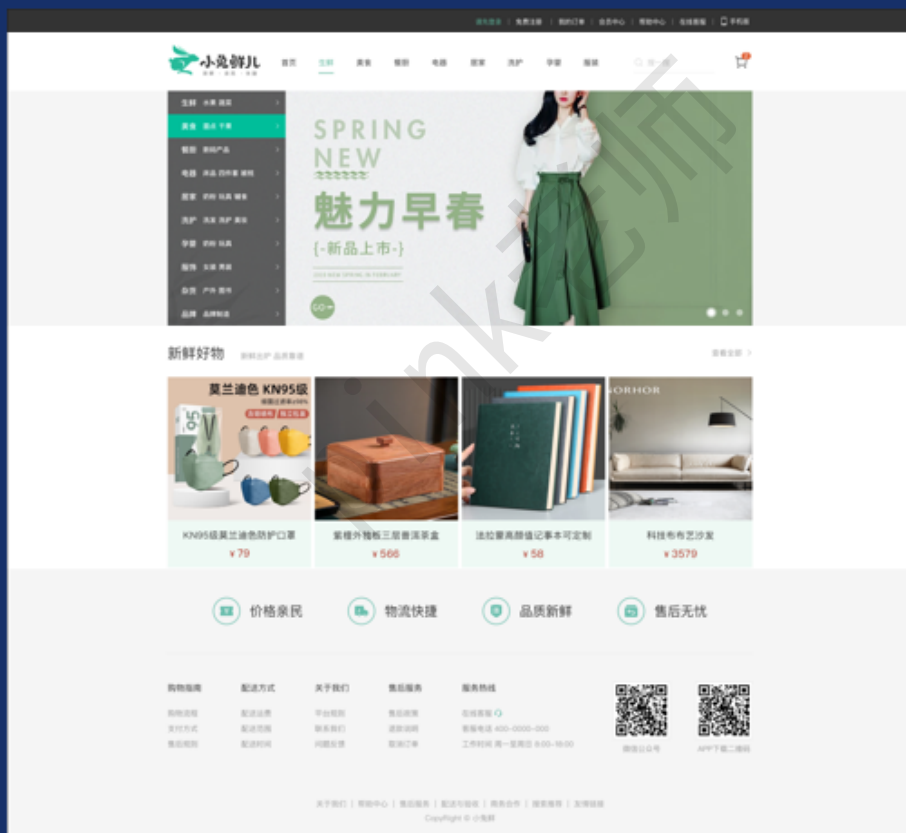
1. 设计稿很重要，可以提高我们开发效率。



小兔鲜实战总结

are you ok ~

2. 对布局理解更深刻了。先整体自上而下标准流布局。



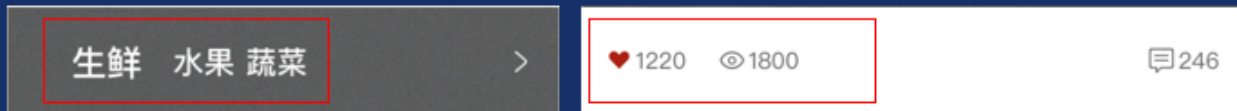
小兔鲜实战总结

are you ok ~

3. 对布局理解更深刻了。大部分盒子一行显示，主轴两端对齐，交叉轴居中对齐。



4. 如果遇到以下情况，注意嵌套盒子，然后再两端对齐。



小兔鲜实战总结

are you ok ~

5. 根据相关性进行盒子拆分。

购物指南

配送方式

关于我们

售后服务

服务热线

购物流程

配送运费

平台规则

售后政策

在线客服 

支付方式

配送范围

联系我们

退款说明

客服电话 400-0000-000

售后规则

配送时间

问题反馈

取消订单

工作时间 周一至周日 8:00-18:00



微信公众号

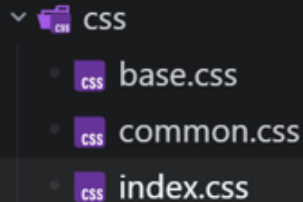


APP下载二维码

小兔鲜实战总结

are you ok ~

6. 公共样式的可以拆分单独css，甚至单独的类。



```
/* 单行文本溢出显示省略号 做一个类名 */  
.ellipsis {  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
}
```

```
<h3 class="title ellipsis">男女情侣抗菌夜视休闲跑鞋闲跑鞋</h3>
```

小兔鲜实战总结

are you ok ~

7. 给盒子起个类名是个好习惯，能尽少减少样式冲突。

```
<li>
  <a href="#">
    
    <h3 class="title">KN95级莫兰迪色防护口罩你值得拥有</h3>
    <p class="price"> <small>¥</small>79</p>
  </a>
</li>
```

```
<div class="navigation" aria-label="网站导航">
  <ul class="JS_navCtn cate_menu">
    <li class="cate_menu_item" data-index="1" clstag=
      "h|keycount|h|keycount|head|category_01a">
      <i class="cate_menu_icon" style="background:url
        (//img14.360buyimg.com/imagetools/jfs/t1/95664/
        15/40921/296/66d13f7cF7ed5ab7d/bd4d68f33a69c7d
        c.png);background-size:100%"></i>
      <a target="_blank" class="cate_menu_lk" href="/
```

8. 写注释是个好习惯，你好，我也好，大家好才是真的好。

小兔鲜实战总结

are you ok ~

布局并不唯一，能实现就是好的。

多练习，多模仿别人，会让自己技术进步。

小兔鲜实战总结

are you ok ~

还有很多布局我们还做不了，迫切需要学习[定位](#)。

[生鲜 水果 蔬菜](#)
[美食 甜点 零食](#)
[餐厨 厨房用品](#)
[电器 厨房电器 厨具](#)
[居家 家纺 软装 收纳](#)
[洗护 洗发 洗护 美妆](#)
[孕婴 奶粉 玩具](#)
[服饰 女装 男装](#)
[杂货 户外 图书](#)
[品牌 品牌制造](#)

SPRING
NEW

魅力早春

{-新品上市-}

2019 NEW SPRING IN FEBRUARY

GO

秘制外皮五福摩提大福点心
150g/盒
美味西点

¥ 39.99

水果面膜韩国蜂蜜柚子茶

找相似

发现更多宝贝 >

浓情蜜意 心口弥漫

浓情比利时巧克力礼盒装
205克/盒
糖果零食

¥ 120.00

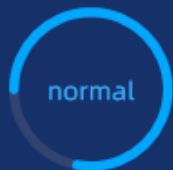
抹茶奶油小蛋糕礼盒装
220克/盒
美味西点

¥ 60.00

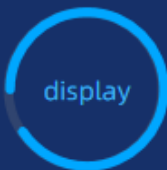
CSS 布局

Finally here~

CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



正常布局



模式转换布局



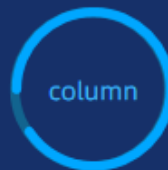
弹性布局



定位布局



网格布局



多列布局

每种技术都有它们的用途，各有优缺点，相互辅助。

- 简单布局：优先使用 Flexbox（一维）或 Grid（二维）。
- 复杂响应式布局：使用 Grid + 媒体查询。
- 文本内容分栏：多列布局（column-count）
- 兼容旧浏览器：浮动布局 或 Flexbox 的降级方案。
- CSS Grid 逐渐成为主流，支持更复杂的布局场景。

定位布局

position~

CSS 定位布局 (**position**) 是控制元素在页面中位置的核心技术之一。通过定位, 可以实现元素脱离文档流、层叠、固定在特定位置布局效果 (定位跟位置有关)

场景:

1. 固定导航栏。页面滚动时导航栏始终**固定**在视口顶部。
2. 吸顶效果。元素在滚动到某个位置后固定。
3. 弹出菜单/下拉框。鼠标悬停时显示浮动菜单。
4. 悬浮效果。元素悬浮在其他元素上方。

.....

定位分类:

相对定位

绝对定位

固定定位

粘性定位



相对定位

die~ father~

CSS 相对定位 (`position: relative`) 是布局中常用的定位方式，其核心在于元素相对于自身正常位置进行偏移

场景：

和绝对定位搭配使用。主要作用是给绝对定位当爹的。

语法：

`position: relative;` 设定元素为相对定位

- 相对于自身原来位置移动距离。
- 不脱离正常流，元素原位置仍被保留，其他元素按原布局排列。
- 可以通过 `top`、`bottom`、`left`、`right` 属性进行偏移。
- 优先级：若同时设置 `top` 和 `bottom`，仅 `top` 生效；同理 `left` 覆盖 `right`。

```
.box {  
  width: 100px;  
  height: 100px;  
  background-color: #pink;  
  position: relative;  
  /* 相对定位 */  
  top: 50px;  
  /* 上移 50px */  
  left: 50px;  
  /* 左移 50px */  
}
```

绝对定位

erzi~ son~

CSS 绝对定位 (`position: absolute`) 核心是脱离正常流并基于定位基准进行偏移。

场景：

1. 弹出菜单/下拉框。鼠标悬停时显示浮动菜单。
2. 悬浮效果。元素悬浮在其他元素上方。

语法：

`position: absolute;` 设定元素为绝对定位

- ❑ 元素脱离正常流，不占据空间，其他元素按原布局排列
- ❑ 相对于最近的已定位祖先元素 (`position` 非 `static`) 移动位置。若都无定位则相对于视口来定位。
- ❑ 可以通过 `top`、`bottom`、`left`、`right` 属性进行偏移。
- ❑ 优先级：若同时设置 `top` 和 `bottom`，仅 `top` 生效；同理 `left` 覆盖 `right`。



定位-子绝父相

erzi~ son~

定位最常用的布局技巧：子绝父相

场景：

1. 弹出菜单/下拉框。鼠标悬停时显示浮动菜单。
2. 悬浮效果。元素悬浮在其他元素上方。



为什么是子绝父相？

1. 子元素可以悬浮其他元素上方，就不能占有位置，影响其他元素布局，所以绝对定位。
2. 但是子元素不能随便乱跑，要配合父元素移动位置，此时父元素需要加定位。
3. 父元素需要占原来的位置，不能影响其他元素布局，此时相对定位最合适。



定位-子绝父相

erzi~ son~

案例：补充小兔鲜常见效果



定位-子绝父相

erzi~ son~

小技巧:



如果用边框, 会比较麻烦:



1. 边框要增大, 需要修改盒子模型为 `box-sizing: content-box;`
2. 盒子变大, 会影响后面小圆点的排列。

如果用阴影更简单:



阴影不占位置, 所以不会增加盒子大小, 从而不影响布局

```
/* 鼠标经过利用阴影实现边框效果 */  
.banner .circle li:hover {  
  box-shadow: 0 0 4px #rgb(255, 255, 255, 0.5);  
  background-color: #fff;  
}
```

定位-子绝父相

erzi~ son~

小技巧：定位的盒子垂直居中



定位-子绝父相

erzi~ son~

小技巧：定位的盒子垂直居中



定位-子绝父相

erzi~ son~

小技巧：定位的盒子垂直居中



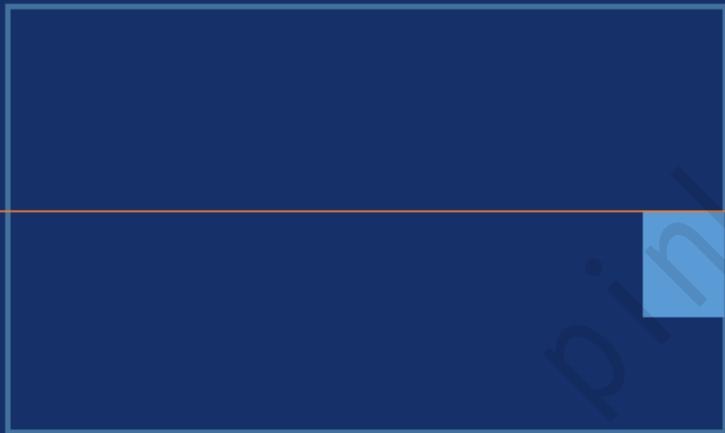
绝对定位： top: 50%;

定位-子绝父相

erzi~ son~

小技巧：定位的盒子垂直居中

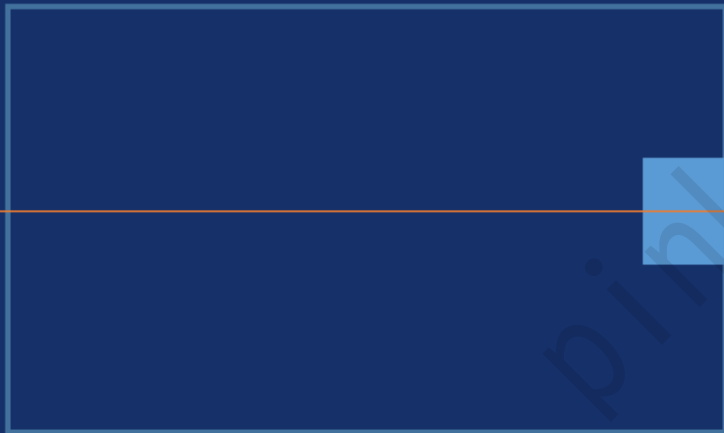
绝对定位： `top: 50%;`



定位-子绝父相

erzi~ son~

小技巧：定位的盒子垂直居中



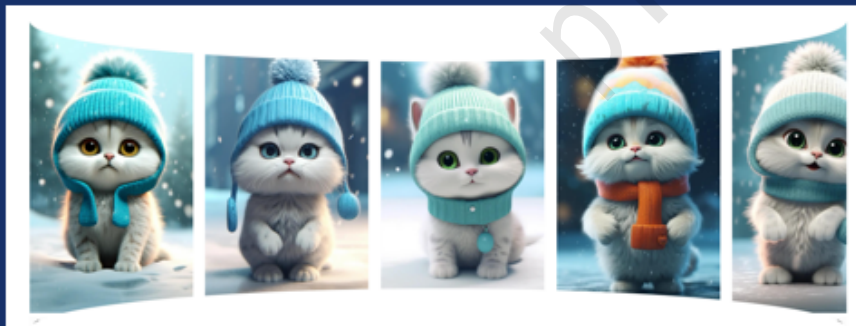
绝对定位： `top: 50%;`

往上走盒子自身高度的一半： `margin-top: -xxpx;`

定位-子绝父相

erzi~ son~

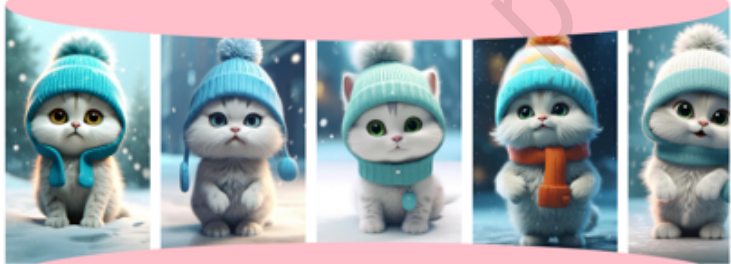
案例：特殊效果



定位-子绝父相

erzi~ son~

案例：特殊效果



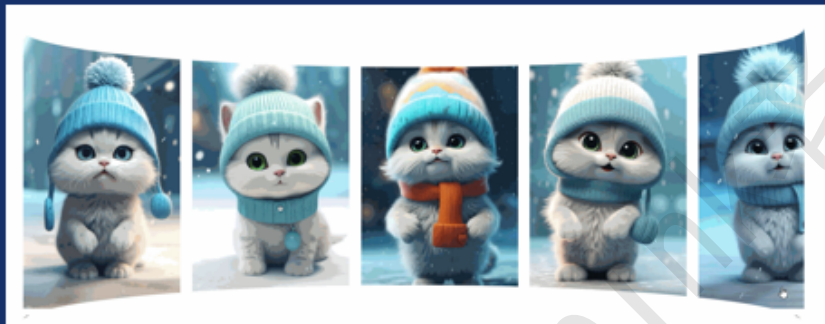
原理：

1. 利用before和after盖住
2. 背景色和网页底色一致

定位-子绝父相

erzi~ son~

案例：特殊效果-添加滚动效果



定位-子绝父相

erzi~ son~

案例：特殊效果-添加滚动效果

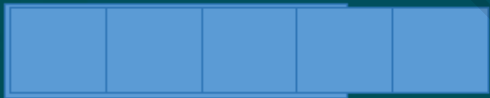
1. 在flex中，width 和 flex: 0 0 200px 区别？

不换行的情况下

1. width: 200px, 所有的子盒子会占据父容器剩余空间，出现压缩情况。
2. flex: 0 0 200px; 子盒子不压缩，每个子盒子宽度就是200px。



width: 200px,



flex: 0 0 200px;

```
.box ul li {  
  /* width: 200px; */  
  /* 不拉伸， 不伸缩 初始值设置为200px */  
  flex: 0 0 200px;  
}
```

定位-子绝父相

erzi~ son~

案例：特殊效果-添加滚动效果

2. 滚动条设置?

```
/* 可以横向 */
```

```
overflow-x: scroll;
```

```
/* 隐藏滚动条 */
```

```
scrollbar-width: none;
```

```
/* 隐藏 WebKit 浏览器 (Chrome/Safari) 的滚动条 */
```

```
.scroll-container::-webkit-scrollbar {  
  display: none;  
}
```

固定定位

sun wukong~

CSS 固定定位 (position: **fixed**) 是一种将元素 脱离文档流并始终相对于浏览器视口定位的布局技术。

场景：

1. 固定导航栏。页面滚动时导航栏始终固定在视口顶部。
2. 页面广告。广告覆盖整个页面。

语法：

position: **fixed**; 设定元素为固定定位

- ❑ 元素脱离正常流，不占据空间，其他元素按原布局排列
- ❑ 始终相对于浏览器窗口（视口）定位，滚动页面时位置不变
- ❑ 可以通过top、bottom、left、right 属性进行偏移。
- ❑ 优先级：若同时设置 top 和 bottom，仅 top 生效；同理 left 覆盖 right。



粘性定位

sticky~

CSS 粘性定位 (position: sticky) 是一种 混合定位模式，结合了相对定位 (relative) 和固定定位 (fixed) 的特性。

场景：

1. 吸顶效果。元素在滚动到某个位置后固定。
2. 表格表头固定。长表格滚动时表头始终可见。

语法：

position: sticky; 设定元素为粘性定位

- ❑ 元素在滚动到指定位置（如 top: 10px）前为相对定位，之后转为固定定位
- ❑ 父容器的 overflow 需为 visible，否则粘性定位失效
- ❑ 可以通过top、bottom、left、right 属性进行偏移
- ❑ 须指定 top, right, bottom 或 left 四个其中之一，才可使粘性定位生效

今日行情

股票代码	股票名称	现价	涨跌	涨幅
000001	平安银行	10.00	+1.00	10.00%
000002	万科A	20.00	+2.00	20.00%
000003	中国平安	30.00	+3.00	30.00%
000004	中国石化	40.00	+4.00	40.00%
000005	贵州茅台	50.00	+5.00	50.00%
000006	格力电器	60.00	+6.00	60.00%
000007	招商银行	70.00	+7.00	70.00%
000008	农业银行	80.00	+8.00	80.00%
000009	建设银行	90.00	+9.00	90.00%
000010	中国银行	100.00	+10.00	100.00%

定位布局-层叠顺序

z-index~

z-index 属性用于控制元素在 三维空间中的**层叠顺序**（即 Z 轴方向）。当多个元素在同一个平面（如同一行或列）重叠时，z-index 决定哪个元素显示在最上层。

场景：

1. 提高层级悬浮效果。
2. 实现图片层叠效果。

语法：

z-index: 数值; 设定层级

- 值类型：整数（正数、负数、零）或 auto。数值越大，层级越高
- 默认值：auto（等同于未设置，后出现的元素覆盖前者）。
- 生效条件：仅对 定位元素（position 值为 relative、absolute、fixed 或 sticky）有效。



定位总结

summarize~

定位类型	脱离文档流	定位基准	层叠控制	典型场景
static	否	默认文档流	无	普通元素布局
relative	否	自身正常位置	支持	微调元素、子绝父相
absolute	是	最近定位祖先 / 视口	支持	弹窗、下拉菜单、悬浮元素
fixed	是	视口	支持	固定导航栏、返回顶部按钮
sticky	否	视口或滚动祖先	支持	吸顶导航栏、侧边栏固定

定位属性：

- **position**。（static/relative/absolute/fixed/sticky）。
- **偏移属性**。top/right/bottom/left 控制元素偏移量。
- **z-index**。控制层叠顺序，数值越大层级越高。

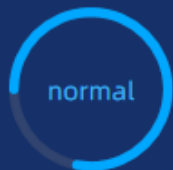
常见组合：

- **固定导航栏**：position: fixed + z-index。
- **弹窗**：position: fixed + 居中技巧 + 高 z-index。
- **吸顶效果**：position: sticky + top。

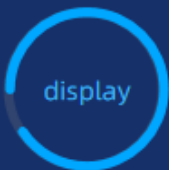
CSS 布局

Finally here~

CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



正常布局



模式转换布局



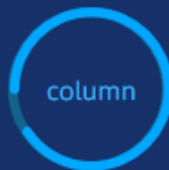
弹性布局



定位布局



网格布局



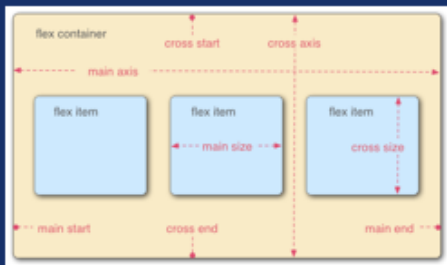
多列布局

每种技术都有它们的用途，各有优缺点，相互辅助。

- 简单布局：优先使用 Flexbox（一维）或 Grid（二维）。
- 复杂响应式布局：使用 Grid + 媒体查询。
- 文本内容分栏：多列布局（column-count）
- 兼容旧浏览器：浮动布局 或 Flexbox 的降级方案。
- CSS Grid 逐渐成为主流，支持更复杂的布局场景。

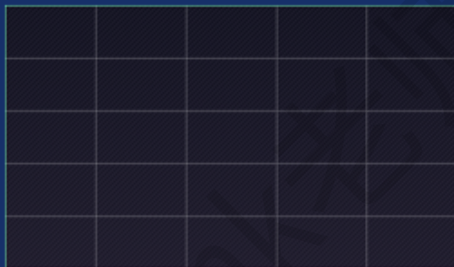
网格布局

grid~



弹性布局 flex

一维布局，只能在单一方向（水平或垂直）上排列元素，适合线性排列的场景。



网格布局 grid

二维布局，可以同时控制行和列的排列，实现真正的二维布局。

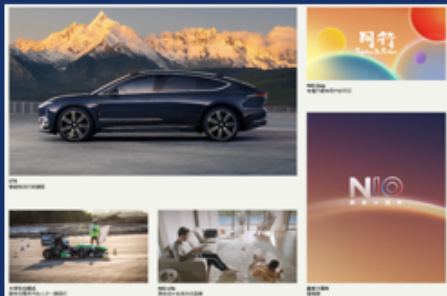
网格布局

CSS Grid Layout

网格布局是一种**二维布局**模型，允许开发者通过定义行（rows）和列（columns）来精确控制网页元素的位置和尺寸。还可以实现**响应式设计**。

网格布局

grid~



网格布局

CSS Grid Layout

网格布局是一种二维布局模型，允许开发者通过定义行（rows）和列（columns）来精确控制网页元素的位置和尺寸。

实际开发大部分是两者混用的：

1. Flexbox：适合快速实现一维布局、动态内容对齐。比如单行布局
2. Grid：适合构建复杂页面框架，提供更强大的二维控制能力。比如多列或者响应式等

网格布局

grid~

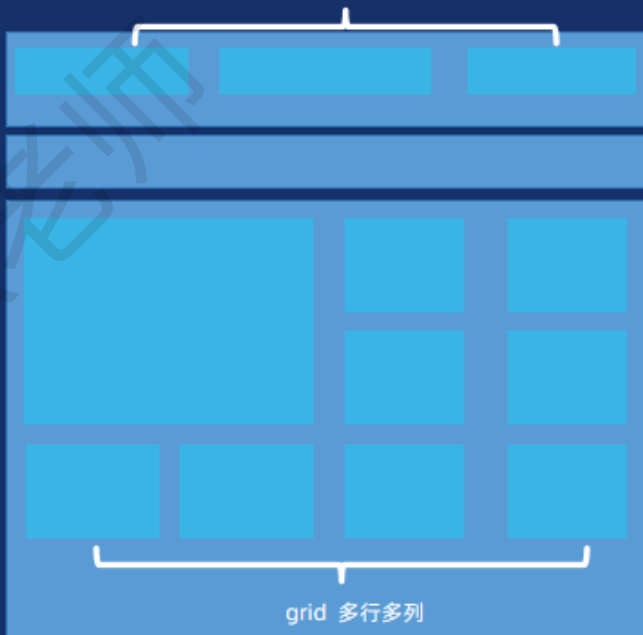
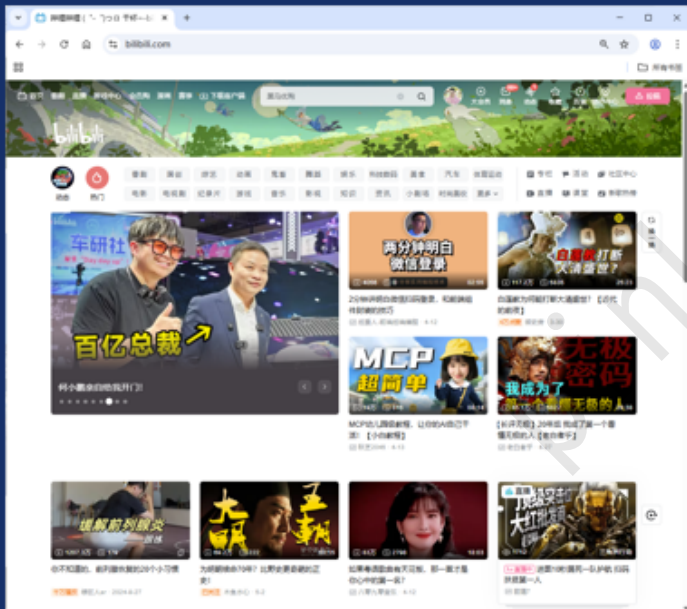
flex 单行

正常流

正常流

正常流

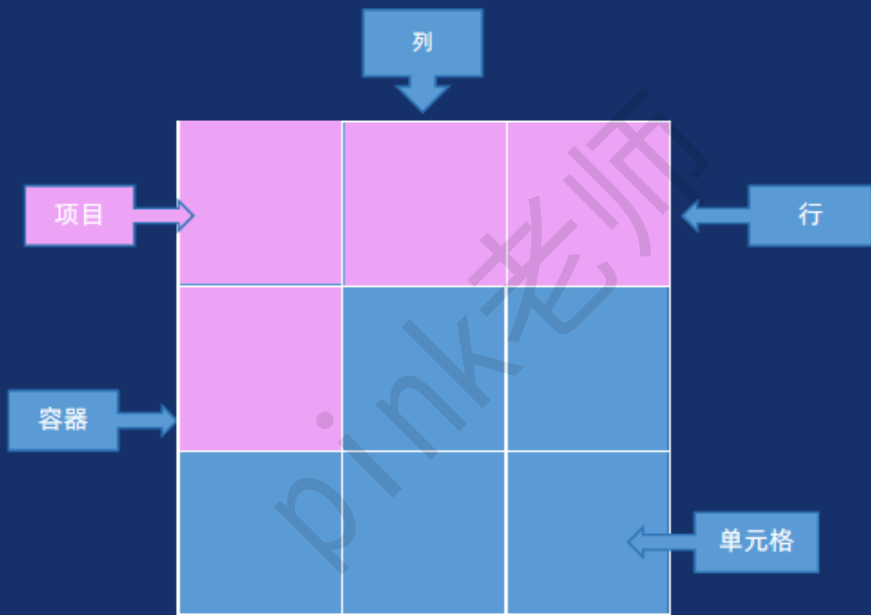
grid 多行多列



1. 网格布局基本概念

you know~

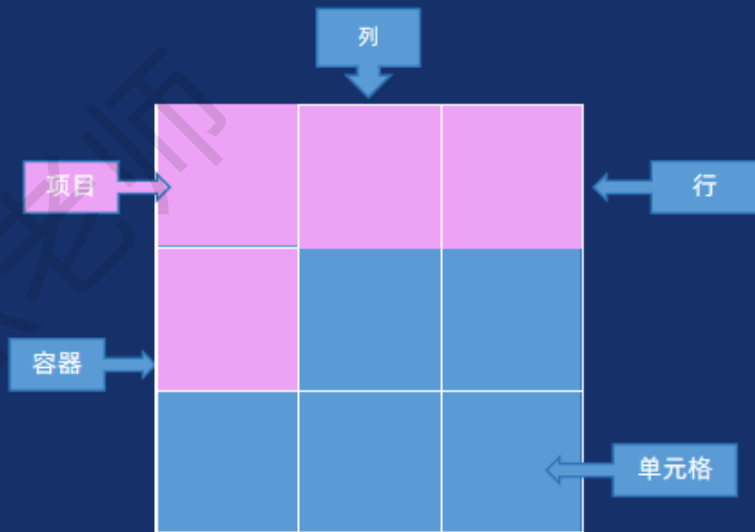
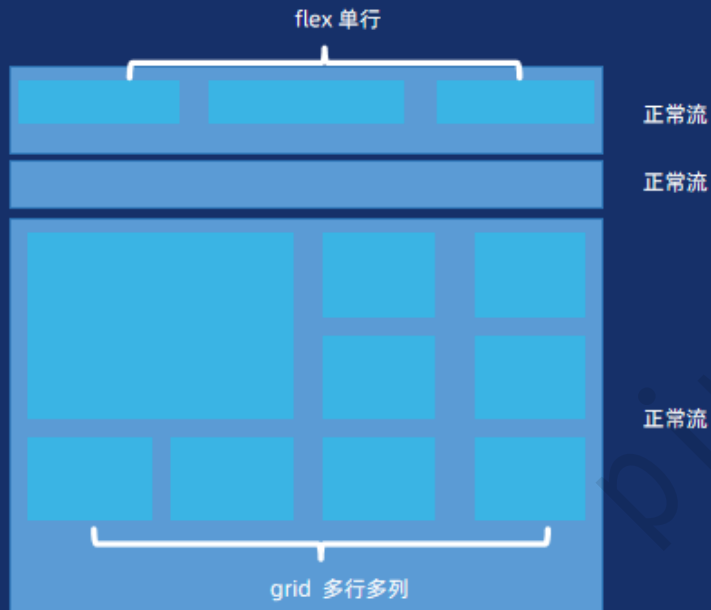
```
<section>
  <div>盒子1</div>
  <div>盒子2</div>
  <div>盒子3</div>
  <div>盒子4</div>
</section>
```



网格布局的核心：创建好网格并放入各类元素

grid 概念总结

you know~



开发常见：单行布局用 flex，多行或者响应式用 grid

网格布局的核心：创建好网格并放入各类元素

2. 网格容器

grid

容器(父盒子)设置 `display: grid;` (块级) 或者 `display: inline-grid` (行内)

与弹性盒子不同的是, 在定义网格后, 网页并不会马上发生变化。因为 `display: grid` 的声明只创建了一个只有一列的网格



3. 网格轨道

grid tracks

网格轨道(Grid Tracks)决定了网格容器的基础布局结构，为子元素提供放置的位置。



绘制网格：(网格轨道)

`grid-template-columns` 定义网格中的列

`grid-template-rows` 定义网格中的行

属性值：

- 有几个属性值代表创建几列/行
- 长度单位 比如 100px

```
grid-template-columns: 200px 200px 200px;  
/* 定义三列，每列宽度为 200px */  
grid-template-rows: 200px 200px 200px;  
/* 定义三行，每行高度为 200px */
```

4. 网格轨道-对齐方式

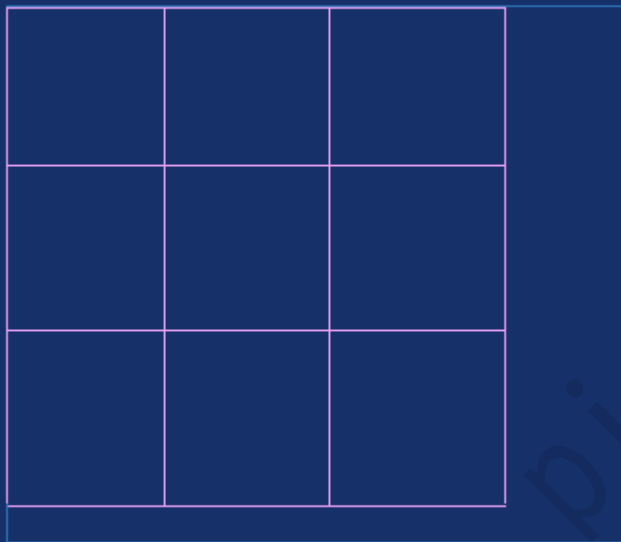
`justify-content`、`align-content`

`justify-content` 是控制列轨道 (Column Tracks) 在容器内水平分布。

`align-content` 是控制行轨道 (Row Tracks) 在容器内水平分布。

属性值	水平方向效果	垂直方向效果
start (默认值)	左对齐	顶部对齐
end	右对齐	底部对齐
center	水平居中对齐	垂直居中对齐
space-around	两侧留出相等的空白, 项目周围空间均匀分布	上下留出相等的空白, 项目周围空间均匀分布
space-between	首尾项目贴边	上下项目贴边
space-evenly	项目间、首尾与边界的空白相等	项目间、首尾与边界的空白相等

总结

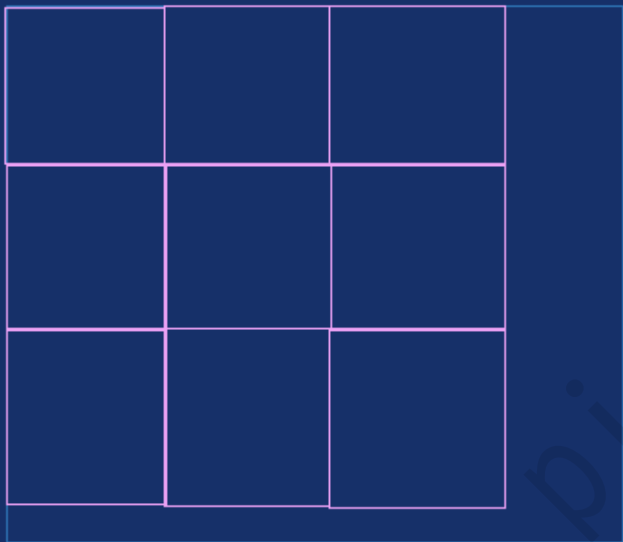


`display: grid;`

创建列轨道：`grid-template-columns: 100px 100px 100px;`

创建行轨道：`grid-template-rows: 100px 100px 100px;`

总结

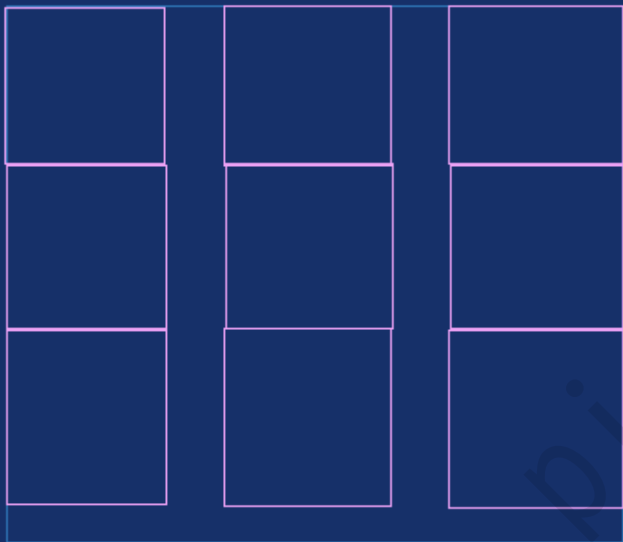


`display: grid;`

创建列轨道：`grid-template-columns: 100px 100px 100px;`

创建行轨道：`grid-template-rows: 100px 100px 100px;`

总结



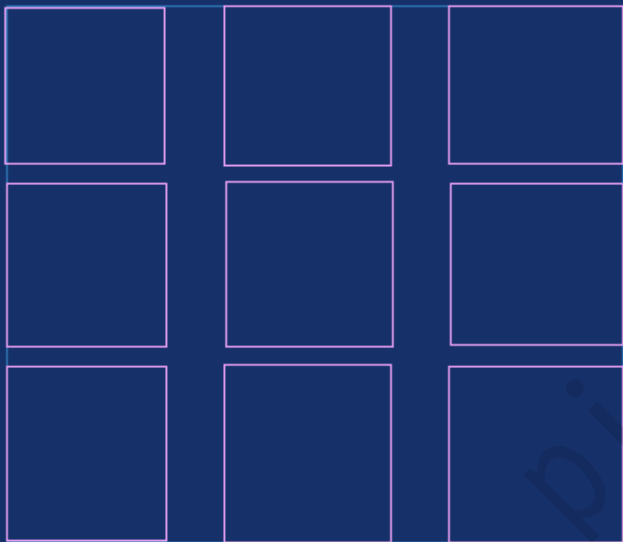
`display: grid;`

创建列轨道：`grid-template-columns: 100px 100px 100px;`

创建行轨道：`grid-template-rows: 100px 100px 100px;`

列轨道两端对齐：`justify-content: space-between;`

总结



`display: grid;`

创建列轨道 : `grid-template-columns: 100px 100px 100px;`

创建行轨道 : `grid-template-rows: 100px 100px 100px;`

列轨道两端对齐 : `justify-content: space-between;`

行轨道两端对齐 : `align-content: space-between;`

3. 网格轨道

grid tracks

绘制网格：(网格轨道)

grid-template-columns / grid-template-rows 属性值。

属性值	说明	示例	应用场景
固定长度	使用 px、em 等固定单位定义列宽	grid-template-columns: 100px 200px;	需要精确控制列宽的固定布局
百分比	按容器宽度百分比分配列宽	grid-template-columns: 30% 70%;	响应式布局中按比例划分列
fr 单位	分配轨道剩余空间的比例 (1fr 表示一份, 总和为容器剩余空间) fr 是 fraction 缩写, 意思 分数	grid-template-columns: 1fr 2fr;	需要自适应比例分配的布局
auto	列宽由内容自动撑开	grid-template-columns: auto 100px;	内容宽度不确定时的灵活布局
repeat() 函数	简化重复的列定义	grid-template-columns: repeat(3, 1fr); (等效于 1fr 1fr 1fr)	多列等宽布局
minmax() 函数	定义列宽的最小值和最大值	grid-template-columns: minmax(100px, 1fr);	响应式布局中限制列宽范围

5. 网格间距

gap

gap 简写属性用于设置行与列之间的间隙（网格间距）

使用场景：

1. 让元素之间保留间隙

番剧

国创

综艺

动画

鬼畜

舞蹈

娱乐

科技数码

电影

电视剧

纪录片

游戏

音乐

影视

知识

资讯

语法：

gap: 20px; 行和列之间保持20像素间隙。

gap: 20px 30px; 行间距是20像素，列间距是30像素。

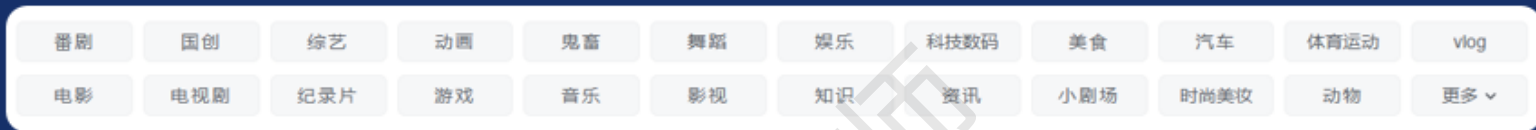
注意： gap是简写形式也可以分开写

column-gap: 30px;

row-gap: 20px;

案例-B站导航条

grid



3. 网格轨道

grid tracks

repeat()函数

语法：

repeat(次数, 轨道尺寸) 或 repeat(自动填充, 轨道尺寸)

2. 自动填充：适用于响应式布局中“列数随容器宽度变化”的场景。

auto-fill：尽可能多地创建列。

auto-fit：尽可能拉伸列填满容器（会合并空白，列宽不小于 minmax 的最小值）

minmax()函数

语法：

minmax(最小值, 最大值) minmax(200px, 1fr)

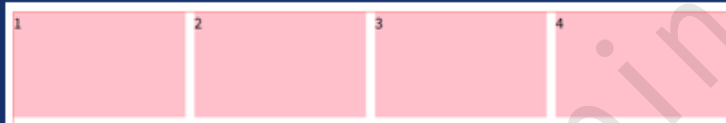
网格轨道

grid tracks

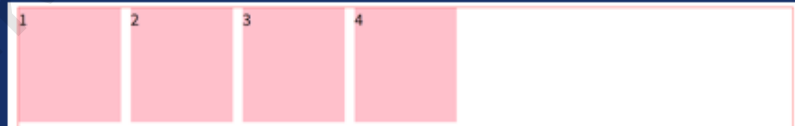
repeat()函数

区别： 当容器空间远远大于所有子元素大小

1. auto-fill 留有空白空间。
2. auto-fit 会拉伸盒子占满父容器空间。



auto-fit

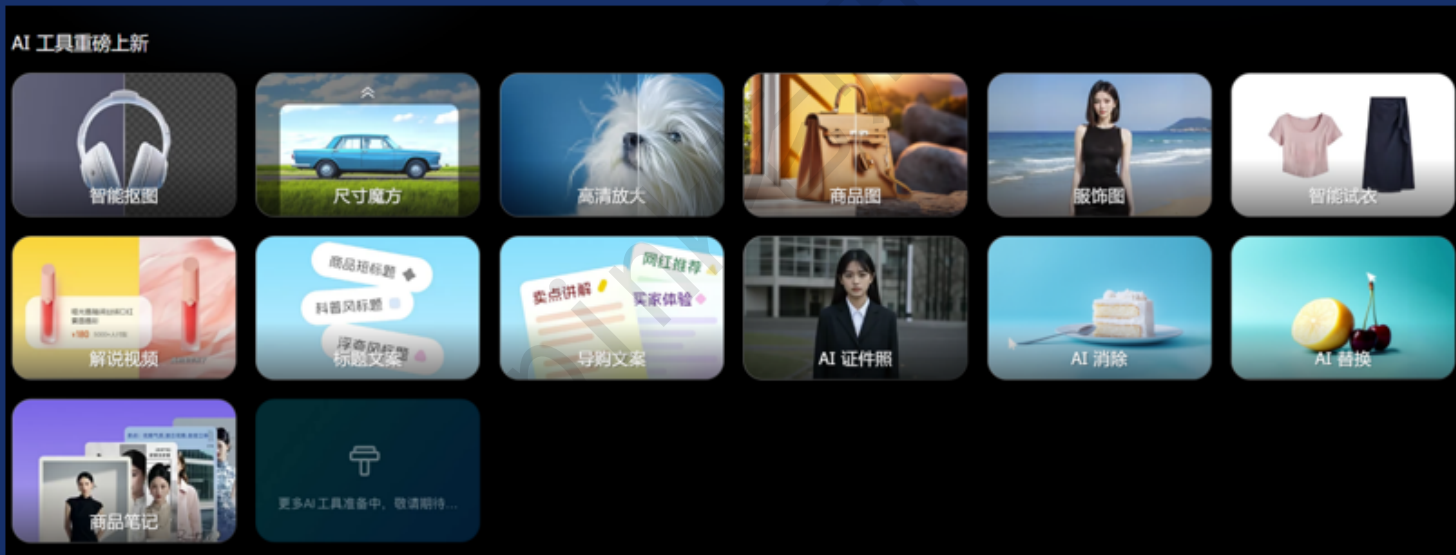


auto-fill

案例- 字体图标响应式

grid

<https://www.iconfont.cn/>



核心代码: `grid-template-columns: repeat(auto-fill,minmax(210px,1fr))`

6. 网格线

grid line~

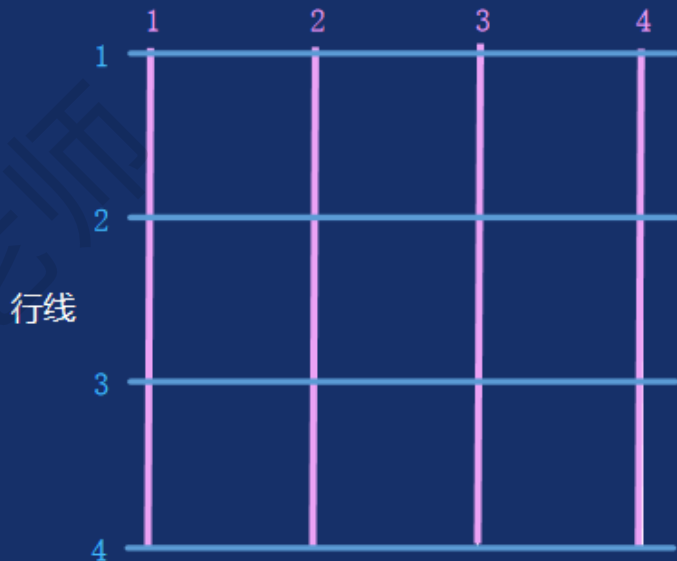
网格线会在使用网格布局时被创建。

使用场景：

1. 实现元素跨越多个网格单元。



列线



线编号从1开始计算，3列就会有4根线

6. 网格线

grid line~

如何实现元素跨越多个网格单元？

实现语法1：

1. 跨列: `grid-column`: 开始线编号/结束线编号;
2. 跨行: `grid-row`: 开始线编号/结束线编号;

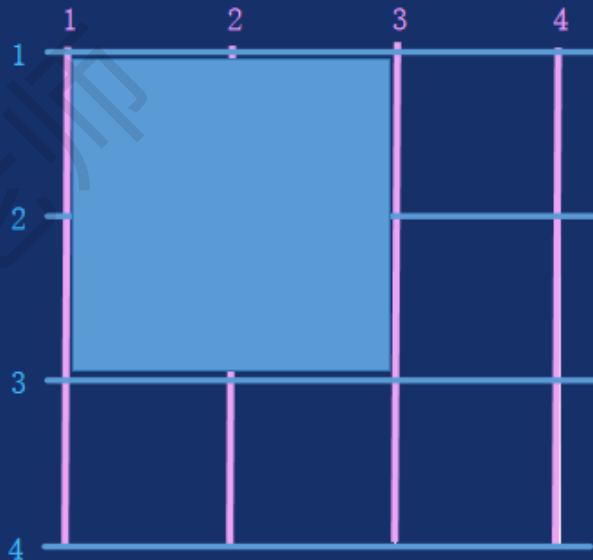
```
background-color: #pink;  
grid-column: 1/3; /* 1/3 表示从第 1 列到第 3 列 */  
grid-row: 1/3; /* 1/3 表示从第 1 行到第 3 行 */
```

注意：

该属性要加到项目（子元素）身上。

列线

行线



6. 网格线

grid line~

如何实现元素跨越多个网格单元？

实现语法2：

1. 跨列: `grid-column: 开始线编号 / span 跨单元格数量;`
2. 跨行: `grid-row: 开始线编号 / span 跨单元格数量;`

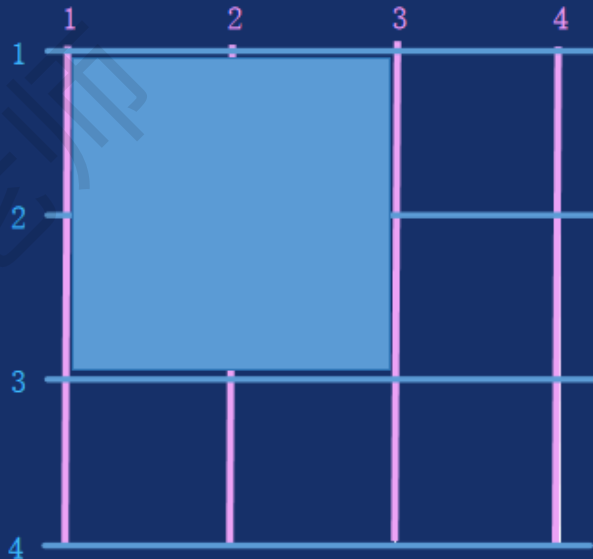
```
background-color: #pink;  
grid-column: 1 / span 2; /* 从1线开始, 跨2个单元格 */  
grid-row: 1 / span 2; /* 从1线开始, 跨2个单元格 */
```

注意：

该属性要加到项目（子元素）身上。

列线

行线



案例-B站分栏



6. 网格线-小游戏网站

grid line~

<https://cssgridgarden.com/>



7. 网格填充方式

grid-auto-flow

grid-auto-flow 决定网格容器中子元素排列填充方式

语法：

1. grid-auto-flow: row; (默认)

子元素按先行后列顺序填充，优先填满一行后换行

2. grid-auto-flow: column;

子元素按先列后行顺序填充，优先填满一列后换列

场景：

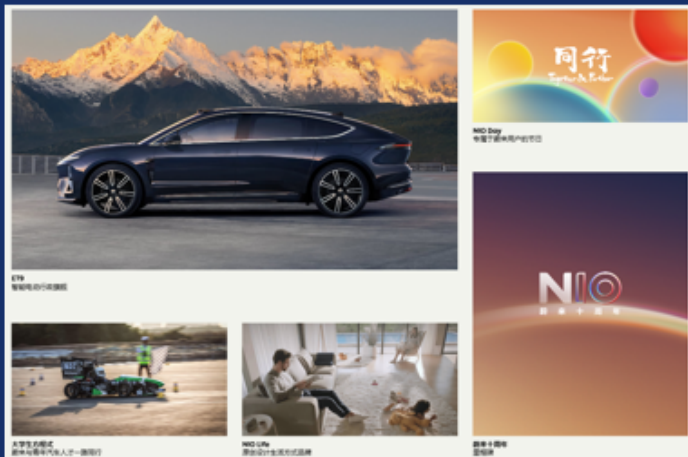
B站、蔚来布局中有些子元素是按照列方式显示。

1	2	3
4	5	6
7	8	9

grid-auto-flow: row;

1	4	7
2	5	8
3	6	9

grid-auto-flow: column;



专栏 活动 社区中心

直播 课堂 新歌热榜

小案例

grid-auto-flow



object-fit

`object-fit` 是 CSS 中用于控制替换元素（如 `<img>`、`<video>`、`<iframe>` 等加载外部资源的元素）内容如何适应其容器尺寸的属性。

值	描述
fill	默认值。拉伸内容以完全填充容器，不保持宽高比，可能导致内容变形（最常用但需谨慎）。
contain	保持内容固有宽高比，缩放至完全适应容器内部（内容全部可见），容器可能有空白（类似“适应”模式）。
cover	保持内容固有宽高比，缩放至覆盖容器（内容可能部分被裁剪），容器无空白（类似“填充”模式）。

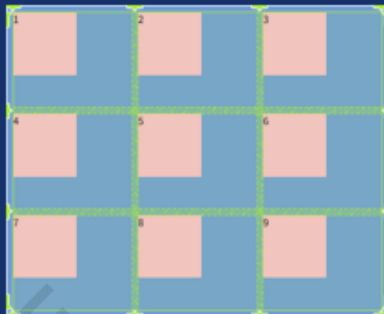
配合：

`object-position`： `object-position` 控制内容在容器内的对齐位置（如 `center` 居中、`top left` 左上角），常与 `object-fit` 搭配使用（例如 `cover` 时调整裁剪区域）。

8.项目对齐方式

`justify-items`、`align-items`

在 CSS Grid 布局中，控制元素在网格内的对齐方式。



语法：

1. `justify-items`: 水平对齐方式; 水平方向（行轴）
 2. `align-items`: 垂直对齐方式; 垂直方向（列轴）
 3. `place-items`: 水平+垂直方式;
- 为了提高可读性和兼容性，提倡使用前面两种方式

场景：

元素在网格内对齐。

注意：

给父容器添加。

300 HOURS OF COURSES

Learn the best tools and platforms

We focus on industry leading platforms so that you can be prepared for your next job. Then we teach all we can about them.

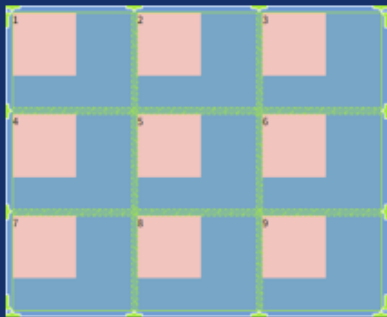


Search

7. 对齐方式

justify-items、align-items

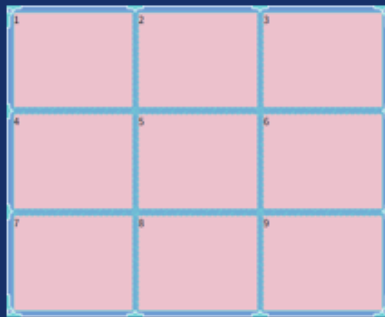
属性值	水平方向效果	垂直方向效果	代码示例
start	对齐单元格起始边缘（左对齐）	对齐单元格起始边缘（顶部对齐）	<code>.container { justify-items: start; }</code>
end	对齐单元格结束边缘（右对齐）	对齐单元格结束边缘（底部对齐）	<code>.container { justify-items: end; }</code>
center	水平居中对齐	垂直居中对齐	<code>.container { justify-items: center; }</code>
stretch	拉伸填满单元格宽度（默认值）	拉伸填满单元格高度（默认值）	<code>.container { justify-items: stretch; }</code>



start



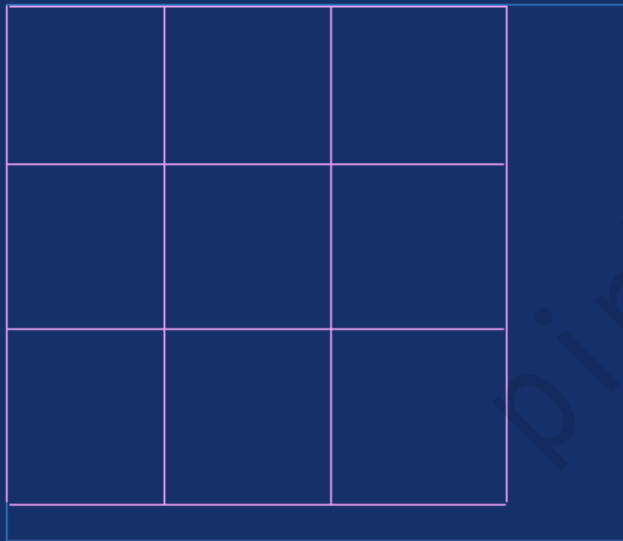
center



stretch 子元素未给大小，则默认拉伸

网格布局-总结

grid



`display: grid;`

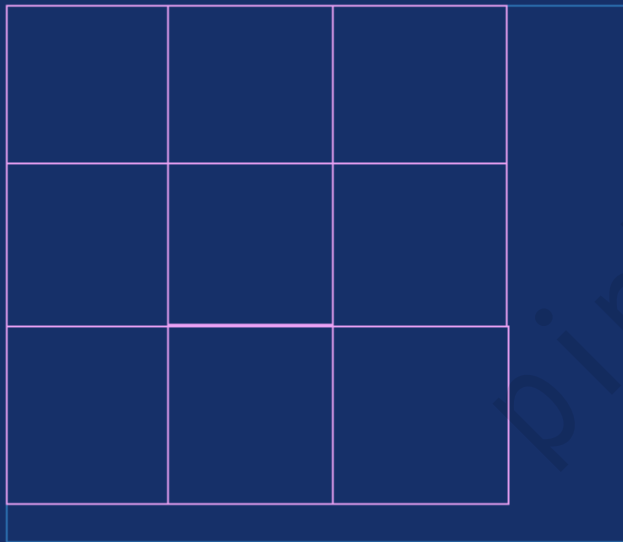
1. 创建网格轨道

创建列轨道：`grid-template-columns: 100px 100px 100px;`

创建行轨道：`grid-template-rows: 100px 100px 100px;`

网格布局-总结

grid



`display: grid;`

1. 创建网格轨道

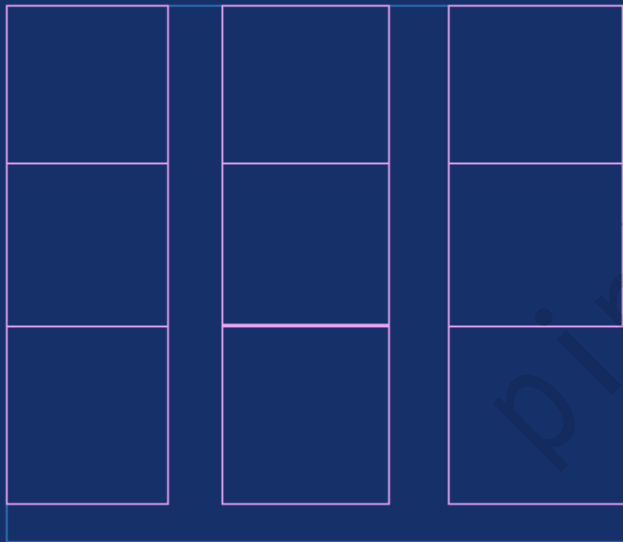
创建列轨道：`grid-template-columns: 100px 100px 100px;`

创建行轨道：`grid-template-rows: 100px 100px 100px;`

列轨道两端对齐：`justify-content: space-between;`

网格布局-总结

grid



`display: grid;`

1. 创建网格轨道

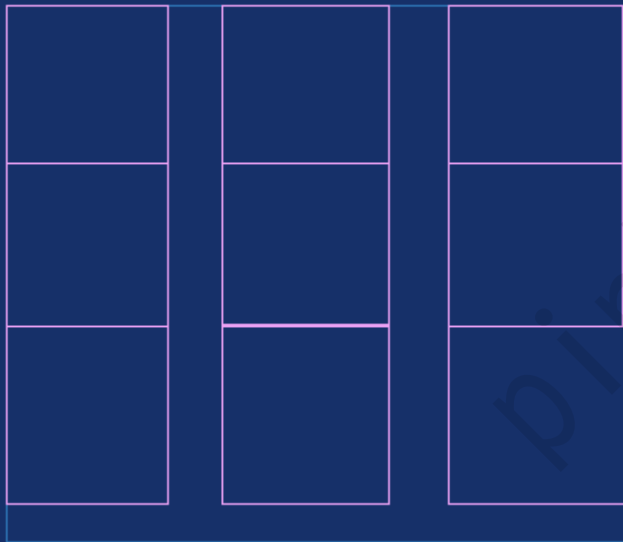
创建列轨道 : `grid-template-columns: 100px 100px 100px;`

创建行轨道 : `grid-template-rows: 100px 100px 100px;`

列轨道两端对齐 : `justify-content: space-between;`

网格布局-总结

grid



`display: grid;`

1. 创建网格轨道

创建列轨道：`grid-template-columns: 100px 100px 100px;`

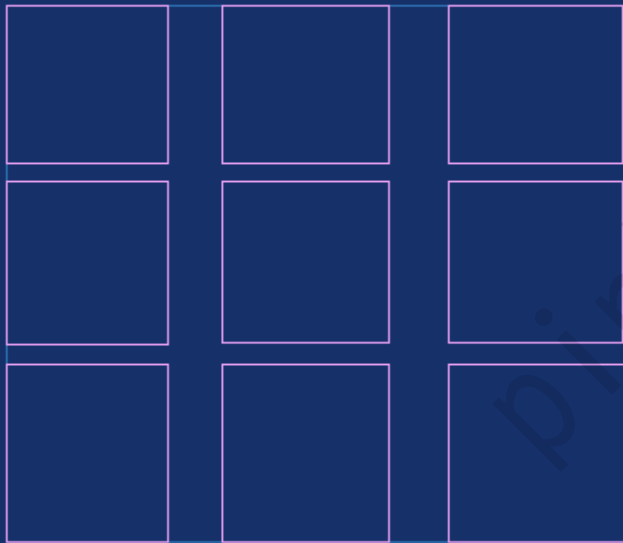
创建行轨道：`grid-template-rows: 100px 100px 100px;`

列轨道两端对齐：`justify-content: space-between;`

行轨道两端对齐：`align-content: space-between;`

网格布局-总结

grid



`display: grid;`

1. 创建网格轨道

创建列轨道：`grid-template-columns: 100px 100px 100px;`

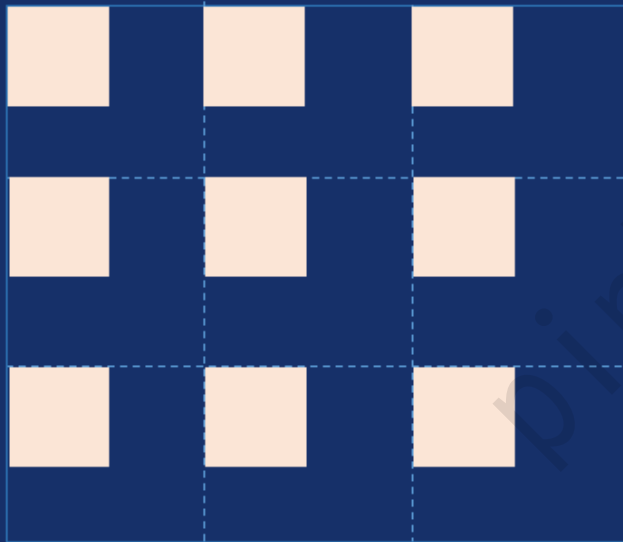
创建行轨道：`grid-template-rows: 100px 100px 100px;`

列轨道两端对齐：`justify-content: space-between;`

行轨道两端对齐：`align-content: space-between;`

网格布局-总结

grid



display: grid;

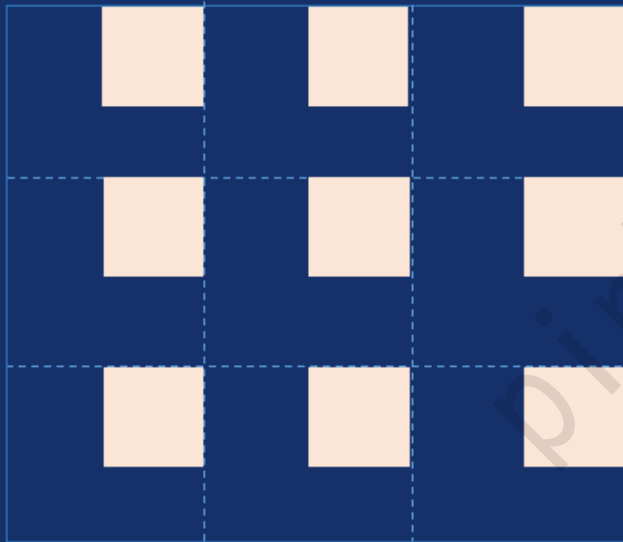
2. 项目对齐方式

水平对齐方式：

1. justify-items: start;

网格布局-总结

grid



display: grid;

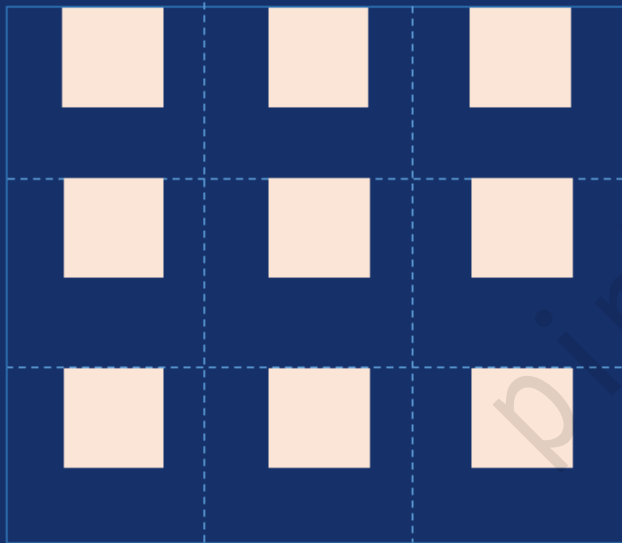
2. 项目对齐方式

水平对齐方式：

1. justify-items: start;
2. justify-items: end;

网格布局-总结

grid



display: grid;

2. 项目对齐方式

水平对齐方式：

1. justify-items: start;
2. justify-items: end;
3. justify-items: center;

网格布局-总结

grid



`display: grid;`

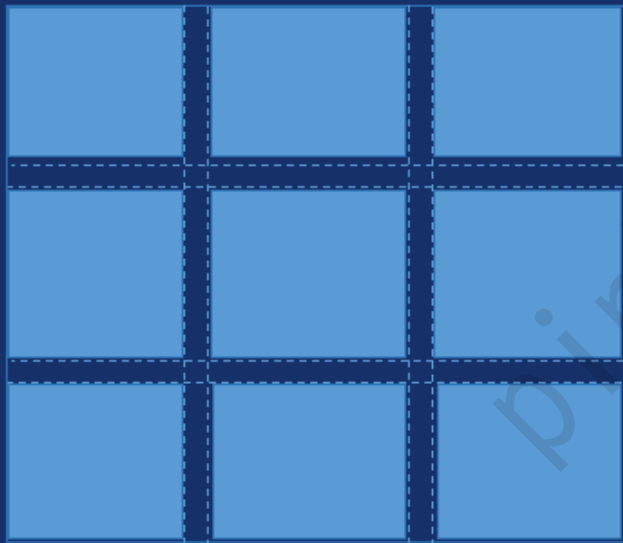
2. 项目对齐方式

水平对齐方式：

1. `justify-items: start;`
2. `justify-items: end;`
3. `justify-items: center;`
4. `justify-items: stretch;`

网格布局-总结

grid



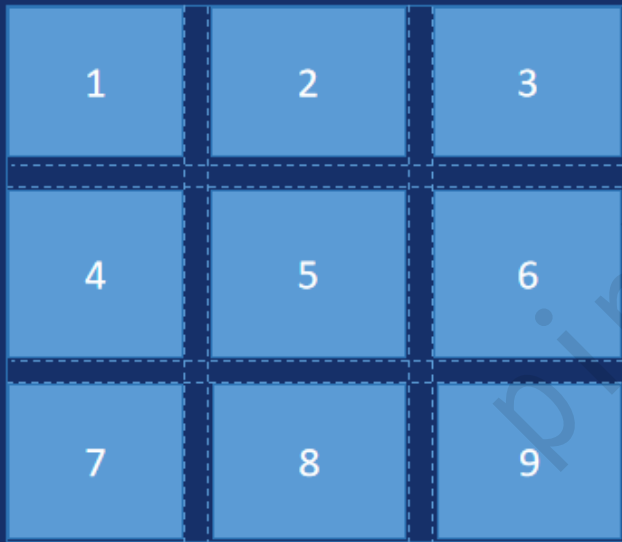
display: grid;

3. gap 网格间隙

gap: 20px;

网格布局-总结

grid



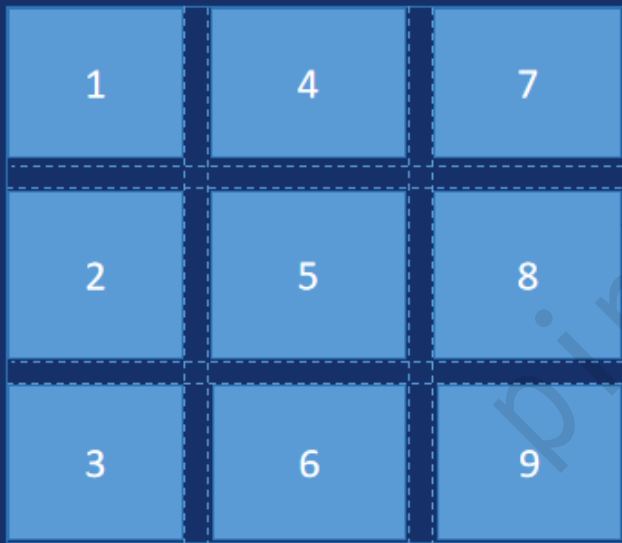
display: grid;

4. grid-auto-flow 自动填充

grid-auto-flow: row; (默认)

网格布局-总结

grid



`display: grid;`

4. grid-auto-flow 自动填充

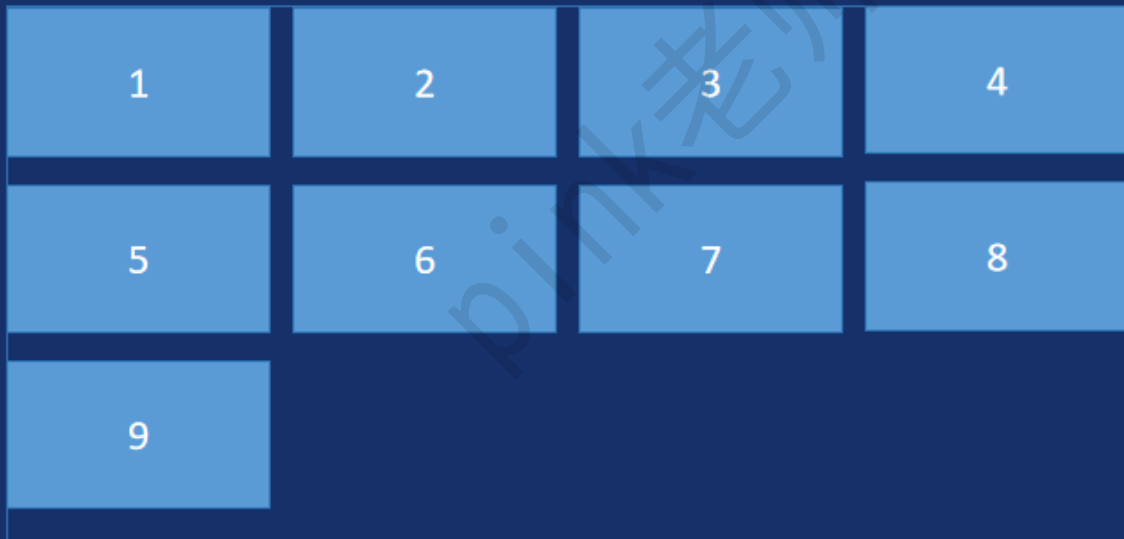
`grid-auto-flow: column;` 竖向

网格布局-总结

grid

5. 响应式布局

核心代码: `grid-template-columns: repeat(auto-fill,minmax(210px,1fr))`



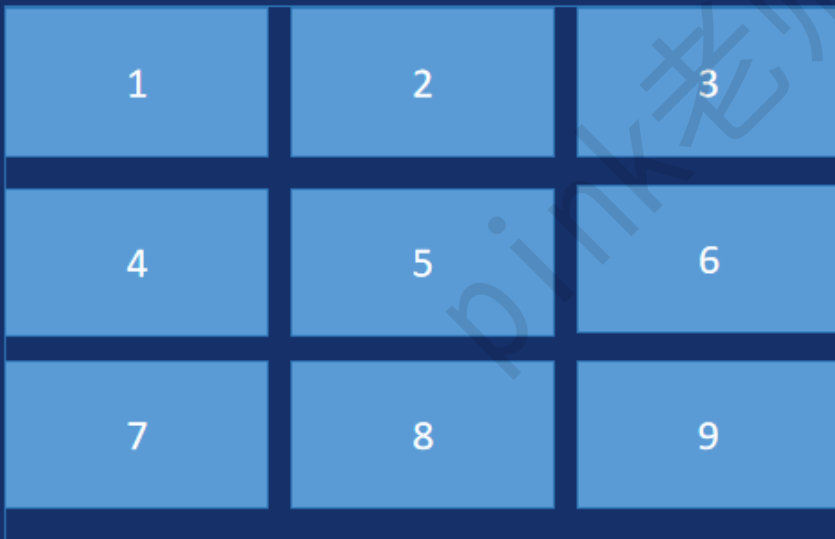
`display: grid;`

网格布局-总结

grid

5. 响应式布局

核心代码: `grid-template-columns: repeat(auto-fill,minmax(210px,1fr))`

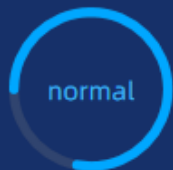


`display: grid;`

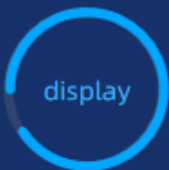
CSS 布局

Finally here~

CSS 布局是网页设计的核心技术之一，用于控制元素在页面中的排列方式。



正常布局



模式转换布局



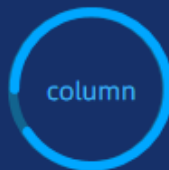
弹性布局



定位布局



网格布局



多列布局

每种技术都有它们的用途，各有优缺点，相互辅助。

- 简单布局：优先使用 Flexbox（一维）或 Grid（二维）。
- 复杂响应式布局：使用 Grid + 媒体查询。
- 文本内容分栏：多列布局（column-count）
- 兼容旧浏览器：浮动布局 或 Flexbox 的降级方案。
- CSS Grid 逐渐成为主流，支持更复杂的布局场景。

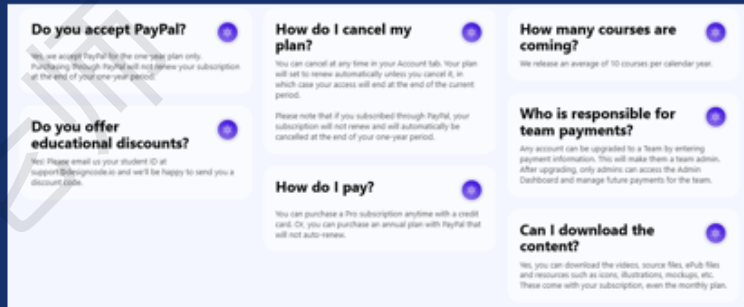
多列布局

Finally here~

多列布局用于将元素的内容 自动分割为指定数量的垂直列

场景:

1. 长文章分栏。文章自动分列，中间有间隙，也可以做响应式。
2. 图片瀑布流。

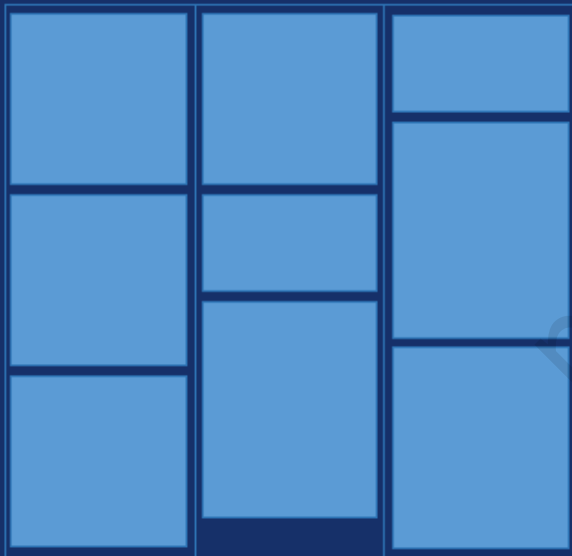


<https://www.yuque.com/>

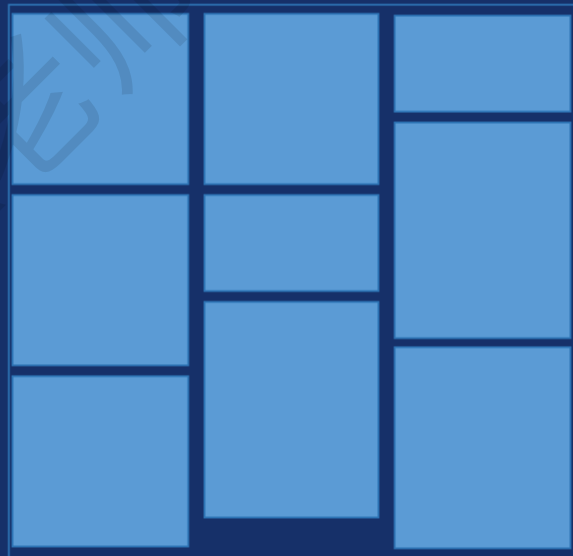
多列布局

Finally here~

多列布局用于将元素的内容 自动分割为指定数量的垂直列



flex或者 grid 布局实现瀑布流（多栏效果）



多栏布局(column)

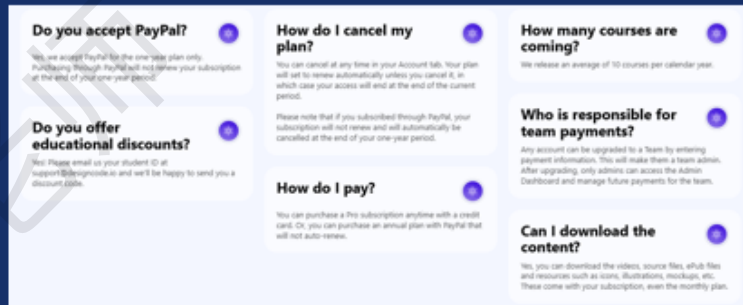
多列布局

Finally here~

多列布局用于将元素的内容 **自动分割** 为指定数量的垂直列

场景:

1. 长文章分栏。文章自动分列，中间有间隙，也可以做响应式。
2. 图片瀑布流。



<https://www.yuque.com/>

属性	作用	示例值
column-count	根据容器宽度和 column-count 值，自动分配列宽	column-count: 3; 分为 3 列
column-gap	列之间的间距	20px、normal (默认 1em)
column-rule	列间的分隔线样式 (颜色、宽度、样式)	column-rule: 1px solid #ccc

多列布局

Finally here~

多列布局用于将元素的内容 **自动分割** 为指定数量的垂直列

注意：

1. 底部子元素被“**切断**”。

原因：column-count的本质是将内容按垂直方向**自动分割到多列**（类似报纸排版），内容会优先填满一列后再流入下一列

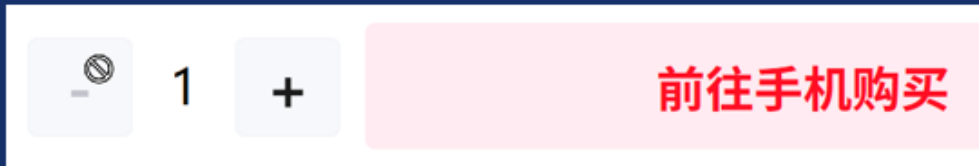
解决方案：

`break-inside: avoid-column;` 强制元素不跨列分割（默认 auto，允许跨列）

给子元素添加。

鼠标样式

在 CSS 中, `cursor` 属性用于控制鼠标指针在元素上的显示样式, 通过改变光标形态可以向用户传递交互提示 (如“可点击”“不可选中”等), 从而提升界面交互体验。



鼠标样式

在 CSS 中，`cursor` 属性用于控制鼠标指针在元素上的显示样式，通过改变光标形态可以向用户传递交互提示（如“可点击”“不可选中”等），从而提升界面交互体验。

值	描述
default	默认箭头（通常是系统默认的箭头光标）。
<code>pointer</code>	手型（指尖朝下的小手），常用于可点击元素（如链接 <code><a></code> 、按钮 <code><button></code> ）。
text	I 型竖线（类似文本输入光标），用于可编辑文本区域（如 <code><textarea></code> 、 <code>contenteditable</code> 元素）。
wait	等待（旋转圆圈或沙漏），表示操作进行中（如加载、提交时）。
help	帮助（带问号的箭头），提示用户需要帮助（如提示信息区域）。
<code>not-allowed</code>	禁止（圆圈带斜线），表示操作不可用（如禁用的按钮）。
grab	抓取（手型带抓取动作），表示可拖动（如可拖拽的元素）。
grabbing	抓取中（手型收紧），表示正在拖动（配合 <code>grab</code> 使用）。
zoom-in	放大（+号），表示放大操作（如图片预览时）。
zoom-out	缩小（-号），表示缩小操作（如图片预览时）。

CSS书写顺序

合理的属性顺序应遵循「**布局** → **盒模型** → **视觉** → **交互** → **其他**」的逻辑，核心目标是让代码更易读、易维护。
实际开发中可根据项目需求微调，但保持团队**一致性**是关键！

顺序	常见属性列表
布局相关	display (显示类型)、visibility (可见性)、position (定位方式)、top/right/bottom/left (定位偏移)、float (浮动)、clear (清除浮动)、overflow (溢出处理)、z-index (层叠顺序)、clip (裁剪)。
盒模型	box-sizing (盒模型计算方式)、width/min-width/max-width (宽度)、height/min-height/max-height (高度)、margin (外边距)、padding (内边距)、border (边框)、outline (轮廓, 可选)。
视觉样式	color (文本颜色)、background (背景, 含 background-color/background-image等)、font (字体含写)、font-size (字号)、font-family (字体系列)、line-height (行高)、text-align (文本对齐)、text-indent (首行缩进)、letter-spacing (字间距)、white-space (空白处理)。
交互与动画	cursor (鼠标指针)、opacity (透明度)、transition (过渡)、animation (动画)、transform (变换, 如旋转、缩放)。
其他 (特殊/低频)	filter (滤镜)、clip-path (裁剪路径)、backdrop-filter (背景滤镜)、will-change (优化提示)、touch-action (触摸行为)、scroll-behavior (滚动行为)。

T H A N K Y O U

行无业境